



Capstone Final Project

ROCK: Robot Operation for Cave Knowledge

High Temp ROV

Jesus Lopez

Cody Stoops

Eugene Chen

Nazli Kaplankiran

Javier Acosta Martinez

May 25, 2026

Contents

1	Introduction/Rhetorical Context	3
2	Background	3
3	Prior Work	4
4	System Architecture	5
5	Design Methodology	5
5.1	Mechanical Design	5
5.1.1	Drivetrain Selection	5
5.1.2	Chassis	8
5.1.3	Shock Absorbers	10
5.2	Heat Management	12
5.2.1	Active Cooling	12
5.2.2	Passive Cooling	13
5.2.3	Final Design	14
5.3	Power System	15
5.3.1	Battery	15
5.3.2	Multi-Rail Architecture	16
5.3.3	Tradeoffs	16
5.4	Electrical System	17
5.4.1	Sensor Selection	17
5.4.2	Sensor Electrical Design	24
5.5	Embedded Systems	29
5.5.1	Architecture	29
5.5.2	Communication	29
5.5.3	Safety	30
6	Subsystem Integration	31
6.1	Data Flow	31
6.2	Control Flow	31
6.2.1	Industry Standard DB25 Connector	32
6.3	Power Distribution	32
6.4	Key Integration Challenges	33

7	Development Methodology	33
7.1	Mechanical	33
7.1.1	CAD to Prototype to Iteration	33
7.2	Electrical	34
7.2.1	Schematic to Simulation to PCB Design	34
7.3	Embedded	36
7.3.1	Incremental Bring-Up	36
7.3.2	System Bring-Up	37
8	Validation Methodology	37
8.1	Communication Validation	37
8.1.1	Packet Integrity	37
8.1.2	Latency and Packet Loss	38
8.2	Control Validation	38
8.2.1	Motor Response	38
8.2.2	Fail-Safe Behavior	38
8.3	Sensor Validation	38
8.3.1	Compare Readings to Expected Values	38
8.3.2	Accuracy Targets	39
8.4	Power Validation	39
8.4.1	Subsystem-Level Power Tests	39
8.4.2	Remaining Full-Load Validation	40
8.5	System Validation	41
8.5.1	End-to-End Pipeline	41
8.5.2	Runtime and Thermal Testing	41
9	Analytical Methodology	41
10	Conclusion	45
11	Schematics and PCB designs	46
11.1	STM32 Shield and PCB	46
11.2	Barometer Module and PCB	48

1. Introduction/Rhetorical Context

Workers, inspectors, and researchers operating in underground environments are subject to risks from high heat, toxic gases, low visibility, unstable structures, and confined spaces. Current tools available on the market are often costly, ill-suited to prolonged underground use, and fail to deliver reliable real-time data on critical parameters. These parameters include gas concentrations, temperatures, pressure levels, visual conditions, and more. This gap in environmental awareness slows decision-making, complicates safety assessments, and ultimately increases the risk of injury or mission failure.

To address this issue, we propose a low-cost ROV (Remote Operated Vehicle) capable of monitoring high-temperature environments in real time. The system integrates gas, temperature, and humidity sensors alongside front and rear cameras for continuous navigation and environmental assessment. A long-distance CAT6 ethernet cable is used to ensure reliable communication in underground spaces. An adaptable chassis, robust power architecture, and robust heat management ensures reliable operation in 50°C temperature environments with an estimated runtime of one hour.

2. Background

To address the gap in ROV capability, the ROCK team needed to understand what the main risks of underground operation were. Two concerns that the team aimed to address after research was the problem of operation in high temperature environments, and measurement of toxic gasses often found in caves.

Air underground is often contaminated by gasses such as Methane (CH_4), Carbon Monoxide (CO), Carbon Dioxide (CO_2), and Hydrogen Sulfide (H_2S). Some of these gasses are odorless and highly explosive, resulting in high risk for those operating underground. For example, 100 ppm (Parts Per Million) of H_2S can cause coughing, eye irritation, altered breathing, loss of smell within 3 minutes, and drowsiness after 15 minutes. Further exposure can cause throat irritation for 1 hour and death after 48 hours [11]. On the other hand, CH_4 is flammable and explosive with the possibility of easily burning oxygen and igniting at room temperature [12]. In either case, unprepared humans do not fare well when exposed to these conditions without warning.

In addition to the gasses, underground workers can suffer from heat-related injuries. Consistent exposure to high temperatures can cause severe sickness and, in extreme cases, death. The Mining Safety and Health Administration (MSHA) reports the underground mines worksites tend to be an average of 26°C. Since there is little to no airflow underground,

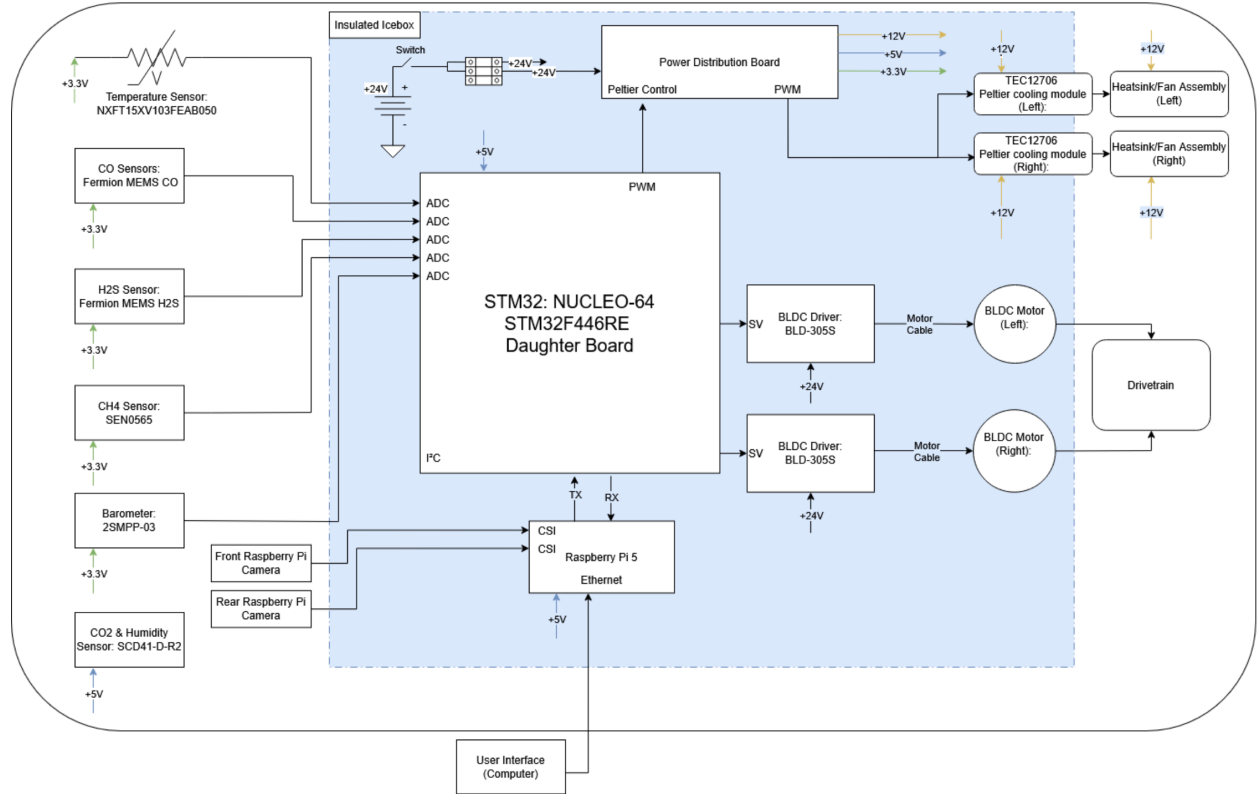
rock builds up heat. At the same time, hot water from deeper below can flow upwards, increasing the temperature of underground walls even further. There are other risks, such as water penetration or stresses degrading the stability of underground spaces, and MSHA additionally describes some of the most common diseases found in workers: heat rash, heat fatigue, heat cramps, heat syncope, heat exhaustion, and heatstroke [13].

Due to the continued depletion of above and near-ground level deposits, the need for clean energy, and the ever-increasing demands for infrastructure, underground operations are expanding and always looking for ways to ensure environmental safety. With the worker's safety being a major concern, the mining industry must develop better tools to reduce the amount of fatal risks.

3. Prior Work

Many robotic systems have been designed for the purposes of data acquisition and increasing worker safety. Bomb disposal robots and reactor maintenance robots are one such example, but high temp underground exploration robots are not common. Oftentimes, researchers and companies will design bespoke systems costing hundreds of thousands of dollars, or otherwise utilize and modify robotic platforms such as the Husky A200[2]. The problem with these off-the-shelf systems is that oftentimes, they lack one or more of the capabilities necessary for underground operations, or are otherwise exorbitantly expensive. The Husky A200, for example, has an automatic shutdown temperature of 50c, where all electronic systems will shut down automatically upon crossing the 50c threshold. Our system aims to fill this gap by providing a low-cost, high-temp, and modular system for cave exploration and data acquisition.

4. System Architecture



A high-level block diagram of the ROV.

The block diagram is a representation of the high level architecture of the whole project. It is components inside of the "ICEBOX" are inside of the blue circle, where as the outside components are mounted on the chassis of the ROV. The block outside of the chassis (big rounded edge box) shows the user interface who is capable of controlling the rover from a safe distance without interaction of the harsh environment the robot may be placed.

5. Design Methodology

a. Mechanical Design

i. Drivetrain Selection

The drivetrain is the part of our rover that is responsible for carrying the chassis weight and also allowing our rover to overcome obstacles such as shallow or steep areas in the road. Several drivetrain designs were researched before deciding on the final version, including one drive and tank drive systems. Before finalizing the drivetrain design, the following subsystem requirements were established:

- Terrain traversal
- Heat tolerance up to 50°C
- Rough terrain
- Minimum 3 in ground clearance
- Shock absorbers with the right length, spring rate, and damping factor in order for the ROV to handle obstacles
- Ability to handle the weight of the chassis (9 kg)
- Independent left and right motor control for different speeds for vehicle motion
- Allow for turning in confined spaces

The design started off as a preliminary rocker bogie idea, but then turned into a tank drive with 6 driving wheels, a following wheel in the front, and a gear at the back. After realizing this was too complicated and after various chassis changes, a triangle-shaped design was adopted, which took the placement of the wheels in the chassis and icebox into account. The triangle design was also more sturdy than the previous tank drive design in that it had a defined center of gravity, similar to a pendulum.

After looking up timing belt sizes, gear research, and Prof. Elkaim’s feedback, it was decided to switch to a capstan drive system. According to Cameron Coward’s coverage of Aead Musa’s robotic dog project, the capstan drive is easy to prototype due to its low cost, high torque compatibility, and offers smooth and quiet operation (Coward, Hackster.io, 2025).

The capstan drive system consisted of four grooves shaped like a capstan drum, with one of them acting as a motor axle extension long enough to pass through the icebox and the chassis wall. The capstan drive operates by a rotating electric motor winding a rope or cable in one direction or the other, forcing the output shaft to rotate accordingly. The mechanism is governed by the capstan equation:

$$T_{hold} = \frac{T_{load}}{e^{\mu\theta}} \quad (1)$$

where T_{hold} is the force the motor needs to apply to the rope to control load, T_{load} is the force needed to move the ROV, θ is the wrap angle of the rope around the drum, and μ is the coefficient of friction. As the wrap angle θ increases, the holding tension required from the motor decreases exponentially relative to the load tension, meaning that more wraps allow

the motor to require less force to turn the wheels. This equation dictates how much wrap is needed on the grooves in order to program the motor controller appropriately.

The drivetrain development process relied on several design methods and strategies working in parallel. Onshape was used for material selection, technical drawings, and generating parts for 3D printing and laser-cutting prototypes. An engineering notebook and MATLAB were used to carry out the shock absorber calculations and the kinetic and potential energy analysis discussed in the following section. Parts such as bearings, sprockets, and rope were sourced through McMaster-Carr, Gobilda, and ACE Hardware after researching appropriate drivetrain components. Design feedback was also gathered throughout the process from Russell Evans, Prof. McGuire, Prof. Elkaim, and Slugworks, which helped shape several of the design iterations described above.

The motor axle sleeve extends from the interior of the icebox to the outside of the chassis wall, where it connects to another groove that drives rotation. In total, the design uses four grooves: three on the wheel sets (three on each set of wheels) and one on the motor axle. These grooves were changed from a sharp spiral design to a capstan drum shape to allow the rope to move more smoothly and reduce slipping. Rectangle cutouts were added to the chassis wall to stabilize the drivetrain mounting.

For the wheels themselves, ABS plastic wheels with a rubber coating were selected for their heat tolerance and suitability for the expected terrain conditions. During prototyping, paracord rope was selected over nylon rope, as paracord provided better friction and grip and could be fused together more easily, making it a cheap and effective option for the prototype. The capstan drum grooves were given a rough texture to further improve grip on the rope. Shaft collar axles were used to enable wheel motion, shock oil was used in the shock absorbers, and a lighter was used to fuse the ends of the paracord rope.

Motor testing was conducted by connecting the BLDC motor to the motor axle sleeve. This testing confirmed that the shock damping worked as intended and that the wheel spirals turned successfully at low speed. However, the testing also revealed trailing arm instability and rope slipping issues. These issues led to a redesign that incorporated the textured capstan drum grooves described above, as well as the addition of bearings on the trailing arms.

A one-wheel drive test was also performed by hand-turning the motor axle. This test showed that more wraps of the rope around the drum resulted in easier rotation, which confirmed the exponential relationship described by the capstan equation. The success criteria for this test, wheel rotation achieved via the capstan mechanism, was met.

ii. Chassis

The chassis is the main body of any robot, existing to support and protect all of a robot's subsystems. Because it interfaces with all other subsystems in this way, design of the ROCK chassis was focused on simplicity, modularity, and ease of assembly.

Ease of assembly was prioritized by designing with flat-pack construction, where all chassis parts were designed to be machined from flat sheets of material and then assembled by hand. The use of retaining pins instead of glue also allowed parts to be swapped in and out at will.



A photo of every chassis component laid flat



Icebox exterior dimensions calculated before CAD modeling

To enable modularity, dimensions for the ROV were split into static, dependent, and independent variables early on. While dimensions such as the size of screws or length of motor were static and unlikely to change, the length, height, width, and material thickness of the ROV walls were expected to change at will based upon subsystem and general system needs. These were independent variables, because they were dimensions that did not directly depend on other dimensions. Dependent variables, such as the size of structural supports, the length of retaining pins, and the distances between certain mounts were considered dependent variables because they needed to be defined in reference to other values. The height of the camera mount, for example, was a dependent variable because it bridged the gap between the base of the chassis and the ceiling of the chassis.

Independent and dependent variables were defined first, before static variables, and when component selection was completed, static variables were set and independent variables were changed to fit the chassis around these static dimensions.

Material selection was a chief concern for multiple reasons, and pugh charts were made to determine what materials should be used for both prototyping and construction. Ultimately, wood was chosen for prototyping due to its low cost, ease of machining, and favorable thermal characteristics. Aluminum was chosen for the final design due to its resistance to deformation, low weight, and high specific heat capacity at $0.89 \text{ J/g } ^\circ\text{C}$.

All of these concerns were shared with the team, and collaborative diagramming on

whiteboards and notebooks were often used to inform design decisions later implemented in CAD, with some early chassis designs being fabricated out of wood for the purposes of informing other aspects of the design from other subsystems.

Due to the flat-pack nature of chassis design, the upper-bound weight of the chassis could be estimated by simply finding the 2D area of expected chassis surfaces, then multiplying it by the material thickness and density of aluminum. A more accurate method used for finding chassis weight is described below.

iii. Shock Absorbers

The shock absorbers were chosen due to the chassis weight and since they work in similar functions such as in RC cars. The weight of the chassis was a huge factor in picking shock absorbers as the weight is important in determining the kinetic and potential energies the shock can supply.

The weight of the chassis was calculated by estimating the total weight of all electrical components from their respective datasheets, as well as the mechanical enclosure by assigning standard aluminum weight on Onshape. The estimated total chassis weight was found to be 8 kg, with the gravitational force being 78.4 N, which meant that the spring force distributed across 6 shock absorbers was 13.0N per shock.

$$\begin{aligned} F_s &= -k\Delta x \\ 6F_s &= F_g \\ F_s &= \frac{78.4 \text{ N}}{6.0} = 13.06 \text{ N} \end{aligned}$$

In order to determine which shock absorber length was the most appropriate for the design, a damping coefficient of 0.4 was chosen, since it allowed for the suspension to bounce but also to recover in a short amount of time. This is important since we want the shock absorbers to recover if they come across a depression or a pothole in the road. The goal is for the potential energy to be greater than or equal to the kinetic energy, since greater potential guarantees overcoming the bump or drop on the road.

Three shock absorber lengths that are all aluminum like our chassis were considered: 1/24 RC 48mm, 1/10 RC 100 mm, and 1/10 RC 120 mm.

Three shock absorber lengths that are all aluminum like our chassis were considered: 1/24 RC 48mm, 1/10 RC 100 mm, and 1/10 RC 120 mm.

The maximum velocity:

$$V_{max} = \sqrt{2gh} \quad (2)$$

where h is the drop height in meters and g is the gravitational acceleration (m/s²).

The damping coefficient:

$$c = 2\zeta\sqrt{Km} \quad (3)$$

where K is the spring constant in meters, m is the chassis weight in kilograms (kg), and ζ refers to the critical damping factor which exists in the 0.2-0.7 range.

The kinetic and potential energies were calculated using the equations:

$$KE = \frac{1}{2}mV_{max}^2 \quad (4)$$

$$PE = \frac{1}{2}k\Delta x^2 \cdot 6 \quad (5)$$

	1/24 RC 48 mm	1/10 RC 100 mm	1/10 RC 120 mm
Extended Length (x_f)	0.048 m	0.100 m	0.120 m
Compressed Length (x_i)	0.027 m	0.072 m	0.082 m
Change in Length ($\Delta x = x_f - x_i$)	0.021 m	0.028 m	0.038 m
Spring rate/constant	610.48 N/m	463.57 N/m	341.58 N/m
Damping Coefficient (c)	55.70 N·s/m	48.54 N·s/m	41.66 N·s/m

Table 1: ROV Shock absorber calculations for each length.

Based on this analysis, the 1/10 RC 120mm shock absorber was selected for the final design. The 120mm option provided the greatest travel distance and the lowest spring rate of the three options considered, which corresponded to the best obstacle recovery performance. The primary tradeoff associated with this choice is that the ROV must slow down when approaching depressions greater than approximately 0.02 m, since the potential energy provided by the 120mm shocks no longer exceeds the kinetic energy of the rover beyond that depth.

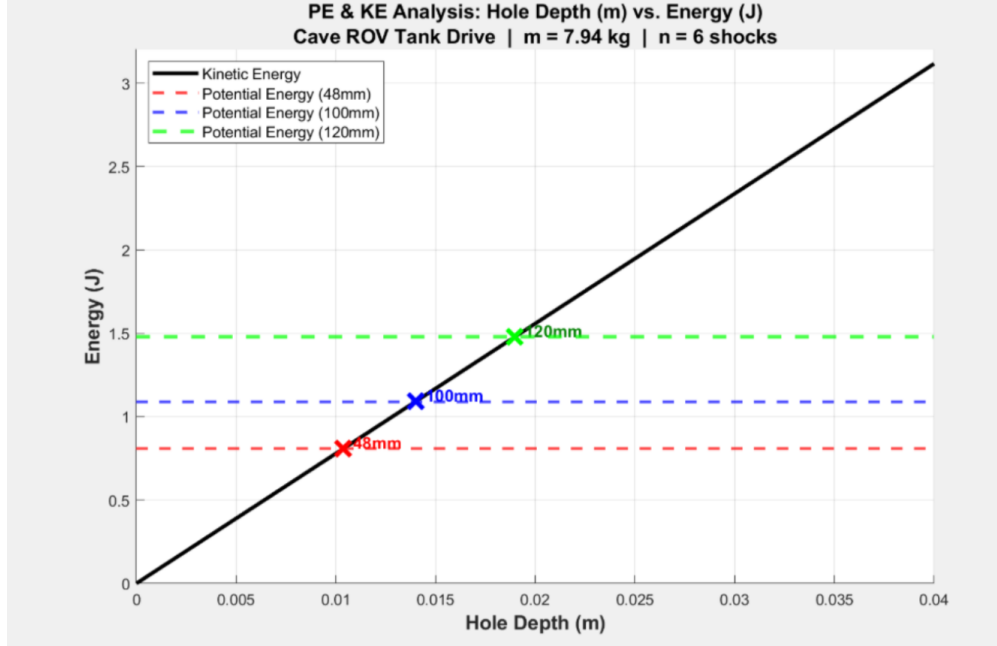


Figure 1: ROV Shock absorber kinetic and potential energy analysis for each shock length.

b. Heat Management

i. Active Cooling

Underground spaces can get extremely hot, and with an explicit design goal of 50c, the ROV needed to have a robust design for heat management. Multiple approaches to active cooling were explored, but ultimately, peltier coolers were chosen as the core of the ROV's active cooling subsystem. Refrigeration units are large, heavy, and expensive; fans can only cool things down to the ambient temperature, and water cooling requires another active cooling system or an inordinately large mass of water to offset collected heat onto.

Peltiers coolers were considered the least bad option given project constraints, with the main advantages of low footprints, low costs, and low complexity fitting neatly into stated objectives. TEC1-12706 peltier coolers costing only 10.80, the main disadvantage was inefficiency and dependence on passive cooling for heat dissipation, which put a strain on the ROV's power system.

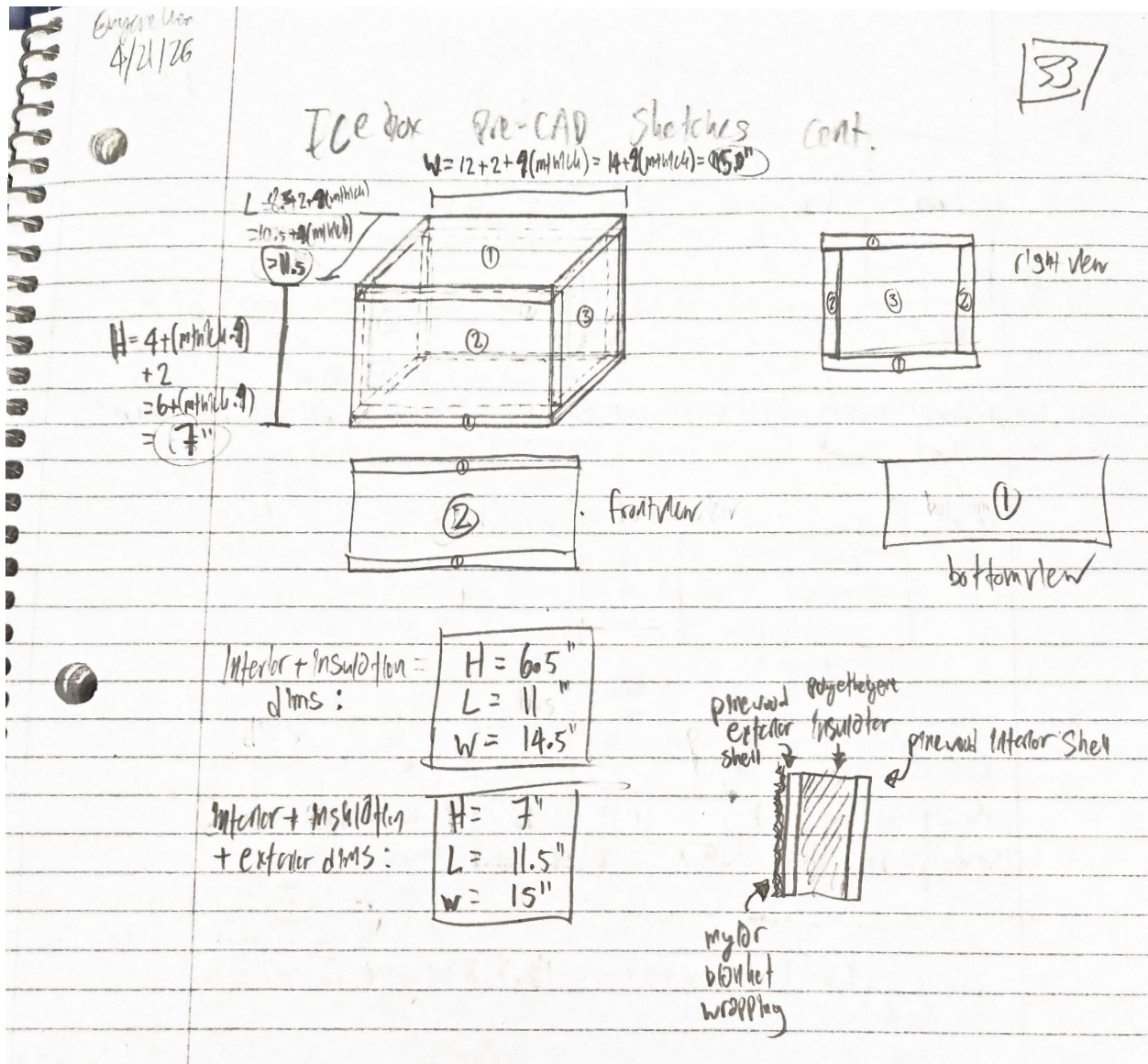
Because of how peltier coolers work, by pulling heat from one side and moving it to the other side of the module, some form of heatsink or active cooling is needed for peltier coolers to work. Given the inability of fans to cool below ambient temperatures, fans could not be used to cool the system as a whole. Because peltier coolers heat up one of their sides above ambient, a fan/peltier combination was considered the best choice available for the project given size and budgetary constraints.

ii. Passive Cooling

Due to the inefficiency of the active cooling methods chosen, passive cooling was an important factor to design for. By reducing the amount of ambient heat peltier coolers had to remove, the ROV could be made lighter, faster, and less power-hungry. given design and budgetary constraints, insulation was the chief method chosen for passive cooling.

The efficacy of insulation is heavily dependent on the size of the space it is insulating, and so a great deal of collaboration was needed amongst electrical, mechanical, and software leads to minimize the space cooled and insulated by the heat-management subsystem. Because the space was cooled by peltiers and had thick insulation, we called this space the icebox.

Given the size of selected components and the space needed for electrical connections between them, with a single motor coming in at a little under 6 inches long, a 12 inch icebox width was determined from the start, and served as a useful starting point for internal layouts. The battery was 6 inches long and 4 inches tall, so when placed against a 2 inch wide motor, the minimum icebox length was determined to be 8.25 inches, to allow for wire routing. As the tallest component within the icebox, the battery's 4 inches was set as the icebox height. With these considerations, the minimum internal volume for the icebox was determined to be 8.25x12x4in (length, width, height), with a 8x6x4 inch volume dedicated to wiring and other components. Because the peltiers and heatsinks sat outside of the icebox, their volumes and dimensions weren't factored into the icebox's internal volume, though their placement on the left and right sides of the icebox were determined early on in the design.



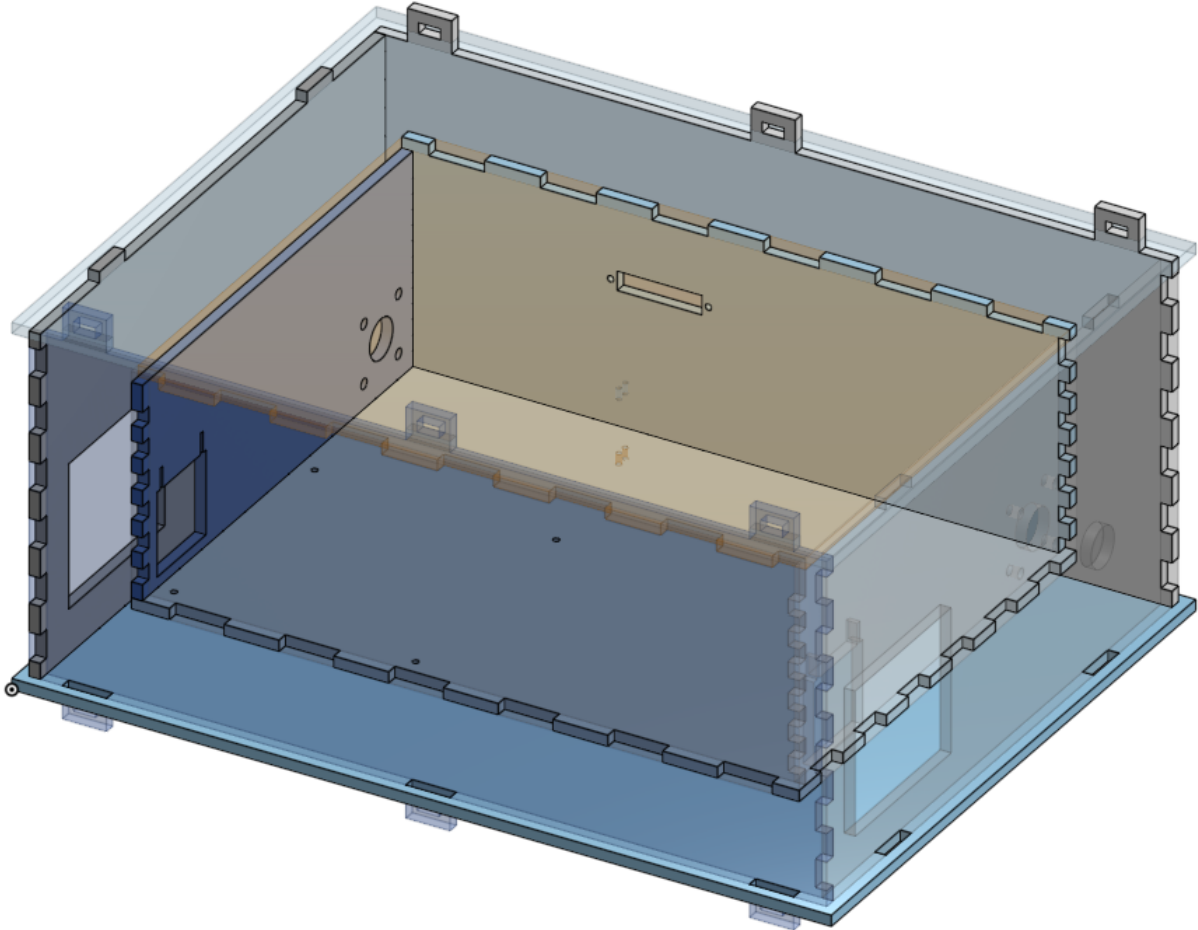
Icebox exterior dimensions calculated before CAD modeling

iii. Final Design

Insulation selection took by far the most amount of work, and was chosen based off of both cost, thickness, and effectiveness. High-performance insulators such as aerogel and heat-resistant polyurethane were favored before icebox internal volume was settled, but these options were ultimately dropped in favor of cheaper, thicker, more easily sourced polyethylene foam insulation available from McMasterCarr.

A 1 inch thick polyethylene insulation was chosen due to its ease of machining and acquisition, though polyurethane insulation remained a viable alternative throughout the design process due to its rigidity, (eliminating the need for an exterior wooden shell) its high

Rvalue (lowering the speed of penetration for heat towards the icebox's internal volume) and its high heat capacity (allowing the system more time before heat began to leech into the icebox's internal volume).



An isometric view of the final icebox design.

The final dimensions for icebox construction was a 8.25x12x4 inch internal volume and an external volume of 11x14.5x7 inch external volume, considering the 1inch thickness of insulation and the .25 inch thickness of both inner and outer icebox wooden framing.

c. Power System

i. Battery

The power system began with the battery and runtime requirements because every downstream regulator, wire path, motor driver, and cooling load depends on the available source voltage and current capability. The rover required a battery that could support mixed loads

from the drivetrain, Peltier cooling system, Raspberry Pi, STM32, fans, and sensors while operating near the target high-temperature environment.

Lithium Iron Phosphate (LiFePO_4) was selected because thermal behavior and safety were more important for this project than maximizing energy density alone. The selected SEFEPODER pack provided a 25.6 V nominal source, which supported the motor-driver input path and provided sufficient headroom for buck-regulated 12 V, 5 V, and 3.3 V rails. The listed pack energy was 230.4 Wh, based on a 25.6 V nominal voltage and 9 Ah capacity.

Battery energy was compared against the power budget to estimate whether the rover could approach the runtime target. For example, if the estimated 1-hour mission energy is 56.141 Wh, then the required battery capacity at 25.6 V is

$$C_{\text{required}} = \frac{56.141 \text{ Wh}}{25.6 \text{ V}} = 2.19 \text{ Ah.} \quad (6)$$

For a 1-hour mission model, this corresponds to an average battery current of about 2.19 A. This value is a power-budget estimate, not a completed full-system runtime result. Full mounted runtime validation was not completed before the final design defense.

ii. Multi-Rail Architecture

The final power architecture separated high-current loads from compute and sensor loads. The 25.6 V battery source feeds the motor drivers and the buck-regulated Power Distribution Board (PDB). The PDB then generates the subsystem rails required by the rover.

The 12 V, 8 A rail supports Peltier cooling and heatsink fan loads. The 5 V, 5 A rail supports the Raspberry Pi 5. The 5 V, 1 A rail supports the STM32. A 3.3 V, 250 mA Low-Dropout Regulator (LDO) supports the sensor loads.

This rail separation was selected to reduce integration risk. Motors and Peltiers create higher-current transients, while the Raspberry Pi, STM32, and sensors require stable low-voltage power. Keeping these rails separated reduces the chance that drivetrain or cooling behavior will disturb computation, reset the controller, or corrupt sensor measurements. The STM32 provides Pulse-Width Modulation (PWM) control signals for the Peltier switching path, but the PDB supplies the actual load current.

iii. Tradeoffs

Several power-system tradeoffs shaped the final design. The first tradeoff was between pre-built regulator modules and a custom PDB. Prebuilt modules were simpler and faster to use, but they would have required separate modules for multiple rails and would have increased wiring complexity. A custom PDB required more design work, layout review, fabrication

planning, and validation, but it created a cleaner architecture for distributing rover power from one battery source.

The second tradeoff was the Peltier switching method. A relay was considered first because it is simple for on/off switching. However, the cooling system needed repeated electronic switching and controllable average power. A low-side N-channel Metal-Oxide-Semiconductor Field-Effect Transistor (MOSFET) was selected instead because it supports STM32-driven PWM control. MOSFET selection focused on current rating, low $R_{DS(on)}$ at low gate voltage, compatibility with a 3.3 V STM32 output, and gate-charge behavior.

The third tradeoff was the sensor rail. A switching regulator is more efficient than an LDO, but a switching regulator also introduces ripple and switching noise. The design therefore used a buck-regulated path followed by a 3.3 V LDO for sensor loads. This preserved a lower-noise supply path for sensitive sensor electronics while keeping the voltage drop and power loss manageable for the expected sensor current.

d. Electrical System

The electrical systems was created with the objective of connecting a controller unit capable of decoding some form of data acquired during runtime. Said data would then be translated into readable output for any controller available in the market. As a result, the project utilizes micro controller units (MCUs) as a way for the R.O.V. to have a "brain". In addition, sensors were chosen as a means to acquire data of gasses and environmental readings in real time.

i. Sensor Selection

Sensors behavior was chosen as a means to acquire data from gasses and environment. In this way, the only concern would be the different types of output that a MCU may need. As a result, the acquired sensors were eight analog output sensors: four for temperature, three for gasses, and one for pressure. Also, one digital output sensor for gas and humidity. In the end, the project would consist of nine sensors in total.

The following are the requirements the team considered when choosing sensors

1. Sensors must withstand the objective ambient temperature of $50^{\circ}C$.
2. The output priority should be analog voltage output.
3. The installation in the chassis should be quick and easy to assemble.
4. Minimal dependency of temperature change.

5. Same interface between sensors for modularity.
 6. gas sensors must represent values of $\pm 10\%$ in reference of external values
 7. environmental sensors must be calibrated to have an offset of $\pm 2^\circ C$, $\pm 2kPa$, $\pm 5\% RH$
- Fermion series gas sensors.

The Fermion series consists of multiple MEMS (Micro-Electromechanical Systems) whose readings are in analog voltage output. These electromechanical devices have an input power voltage up to +5.5V. Since the team focused in integration rather than ground up sensor behavior, these sensors were purchased as modules from "DFRobot" supplier. The supplier offered two different options: An expensive sensor capable of self-calibrating over time and a cheaper version with no calibration (assumed pre-calibrated from factory). Thus, due to the limiting budget, the cheaper option was selected.

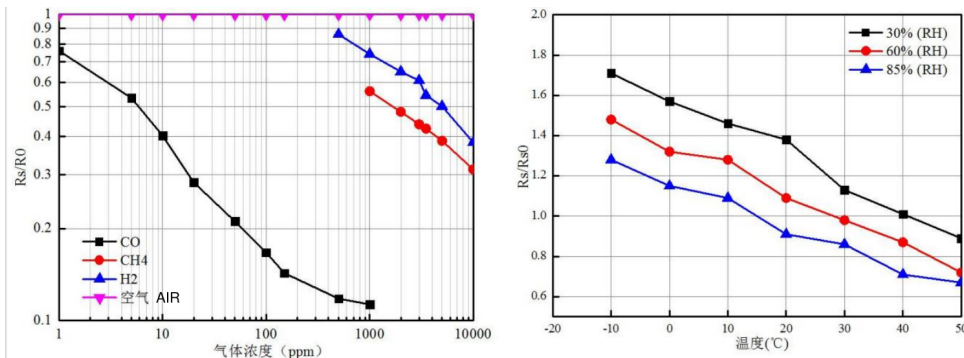
Fermion MEMS sensors were the first chosen. This initiated the sensor interface standard of three pins: Voltage in, Voltage out, and sensor output.

The output behaves similarly as a potentiometer. That is, the output voltage is a divider of resistors; let's call this ' R '. R is the ratio between the impedance when the target gas is not present (R_0) and the impedance value when the target gas is present (R_S). In other words:

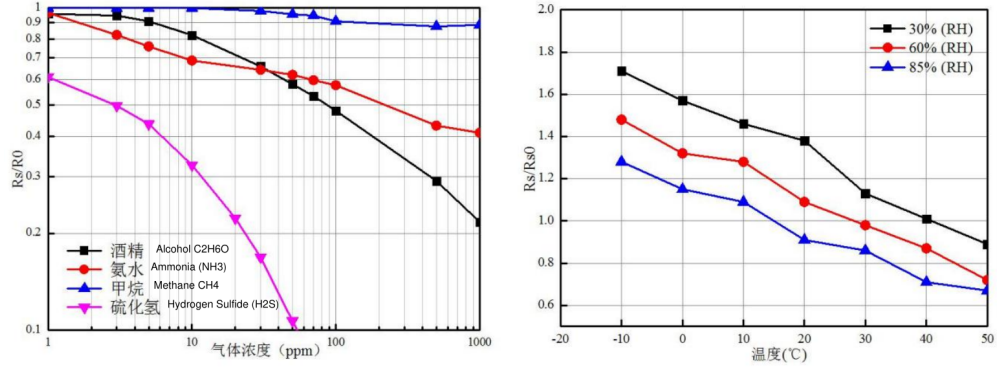
$$V_{out} = IR = V_{in} \frac{R_S}{R_0}$$

Where V_{out} is the output of analog voltage of the sensor, I is the input current of the sensor, V_{in} .

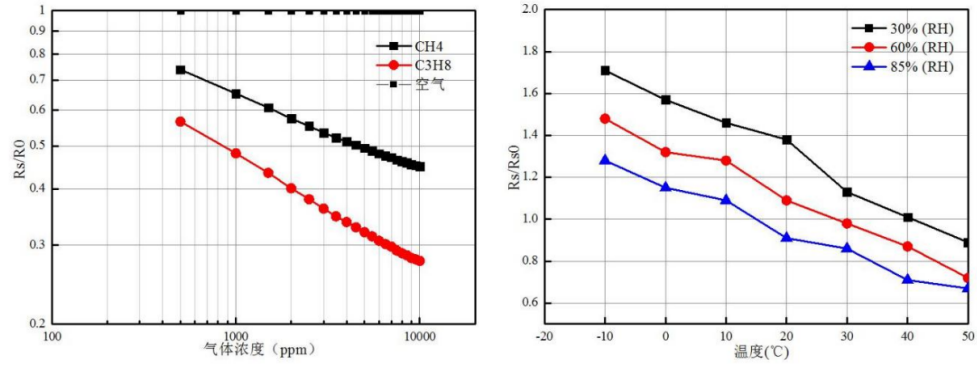
All this information was obtained from the datasheets, whose original language is Chinese. As a result, the team used google file translate option to understand the datasheets. With the translated version acquired, a major disadvantage of this sensor was observed.



ppm values vs R_S/R_0 ratio (left) and temperature dependency vs R_S/R_0 (right) of CO sensor [5].



ppm values vs R_S/R_0 ratio (left) and temperature dependency vs R_S/R_0 (right) of $H2S$ sensor [6].



ppm values vs R_S/R_0 ratio (left) and temperature dependency vs R_S/R_0 (right) of CH_4 sensor [7].

Analyzing the previous graphs, the team noticed three major dependencies on the project: cross-contamination on the left graphs, humidity dependency (%RH), and temperature (°C) on the right graphs. Moreover, the CH_4 sensor does not output consistent ppm values.

In the face of this issues, the team decided it was best to discard the cross-contamination due to not consistent information of natural formation, in underground environment, of those gases would create an issue. Temperature and humidity was mitigated with the decision of adding temperature and humidity sensors as part of the sensor list. Lastly, the CH_4 sensor was decided to become a digital output signal and thus only detecting if the gas is present (no ppm accuracy).

- Sensirion SCD41 sensor [?].

The SCD41 is a three in one package sensor, with the ability of measuring CO_2 , humidity, and temperature. These sensor features a digital output, it follows I^2C communication protocol. Thus, this sensor did not meet requirement #5. This was chosen by the team due to its versatility, price, and information available. This sensor works with a supply voltage of +2.7V to +5.5V.

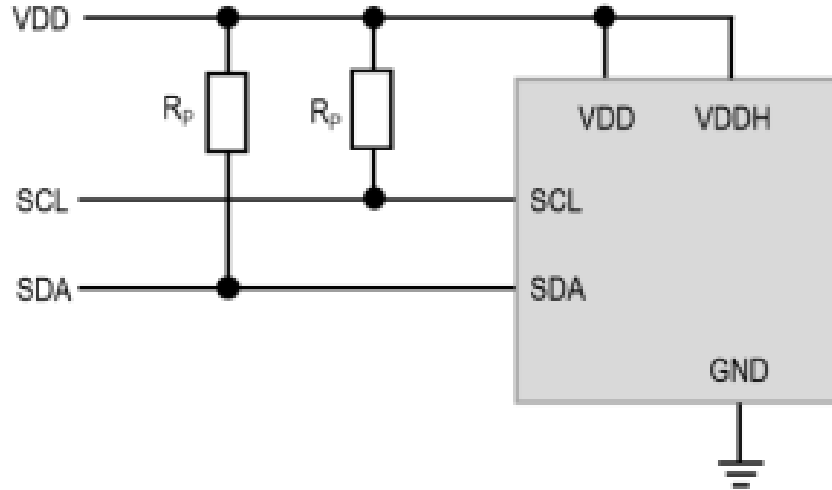
The versatility of the SCD41 has a major disadvantage: time readings. The time periods between readings are different for what output the MCU is requesting. CO_2 readings updates every 60s, humidity updates 90s, and temperature 120s. The gas and humidity readings wait period was accepted by the time, but we rejected the temperature readings. This is due to the critical dependency on temperature the FERMION MEMS sensors need. As a result, the temperature feature was omitted.

Since the output of the SCD41 is given in bytes, the datasheet explains the byte obtained (B) can be used to deduct the current state of CO_2 and humidity by using the following formulas:

$$CO_2[ppm] = B[0]$$

$$RH[\%] = 100 * \frac{B[2]}{2^{16}-1}$$

Note: B[n] refers to the response order (B[0] go first)

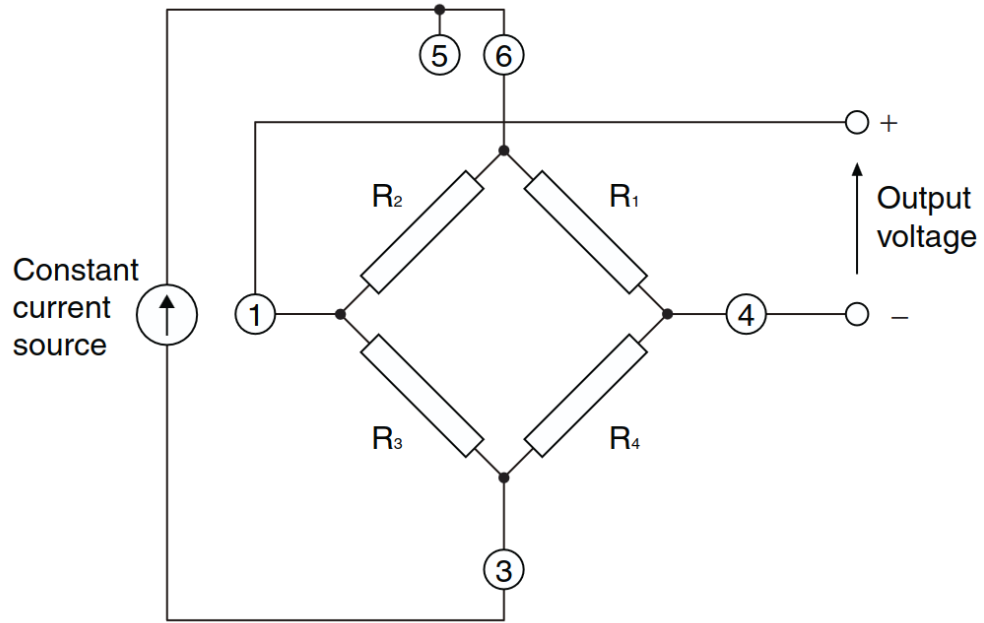


Set up of SCD41 provided by the datasheet in regards of I^2C protocol.

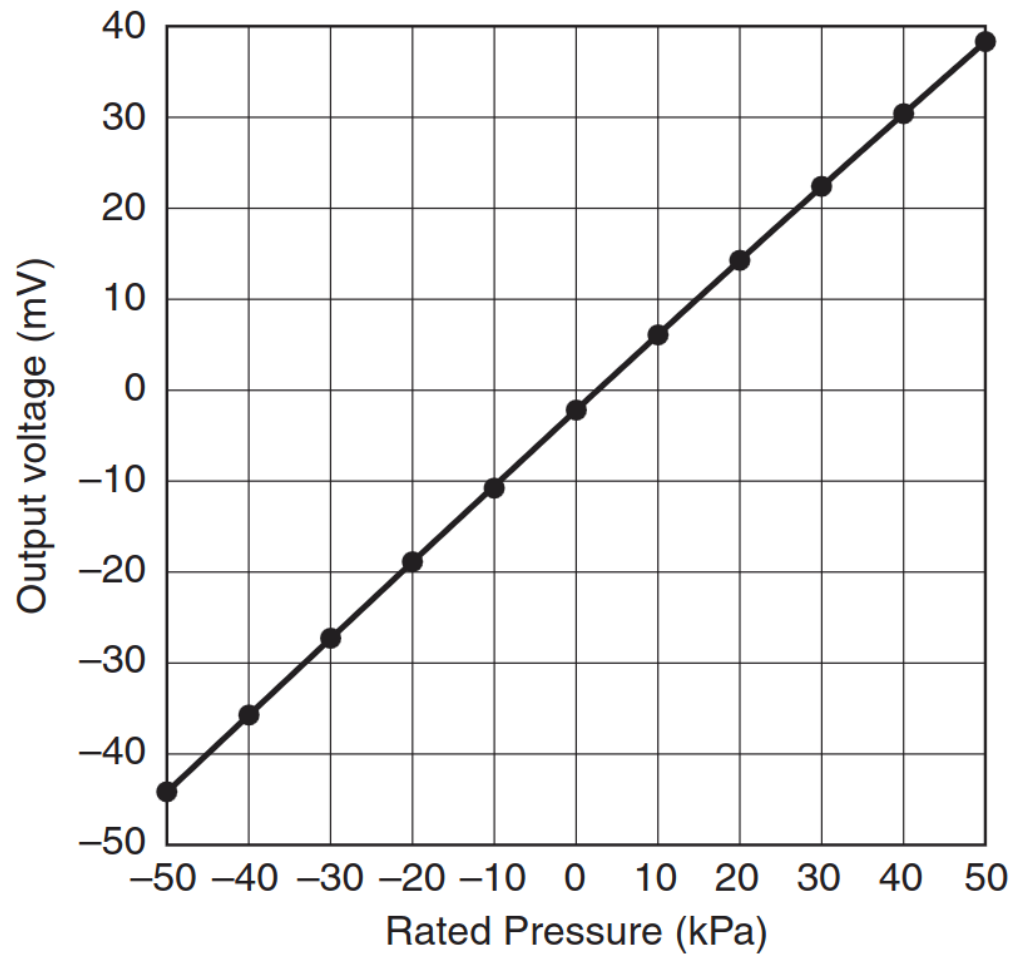
- 2SMPP-03 sensor [8]

the 2SMPP-03 is a barometer (pressure) sensor with the capability of measuring between -50kPa to 50kPa. For this project, the team opted out for measuring the pressure under 0kPa due to this value being a negative value and thus, not complying with the requirements previously stated. Instead, we decided it was best to use the positive pressure as this is more common in underground exploration.

The barometer works similarly as a Wheatstone bridge. This configuration of resistors is designed to output a differential voltage between two pins. Additionally, the barometer demands a $100\mu A$ as the supply power. Thus, this sensor does not meet requirement #5. The design around this issue is explained on the next category.

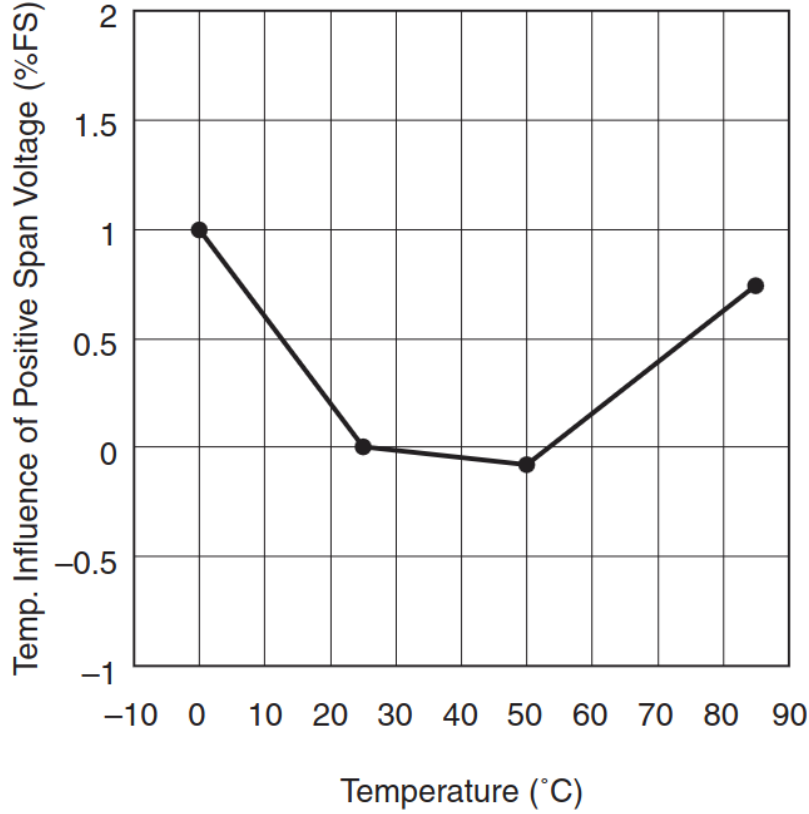


Wheatstone bridge set up from barometer datasheet. The number on the nodes refers to the physical pin designation of the sensor.



Graph of pressure measured vs kPa values. The team does not measure pressure below 0kPa.

Temperature influence of Span voltage (at 0 to 50 kPa)



Barometer differential output voltage dependency of ambient temperature.

The dependency of temperature and pressure for the sensor output leads to the following equations:

$$FS(T) = \begin{cases} -0.04T + 1 & \text{if } 0 \geq T > 25 \\ -0.45(T + 25) & \text{if } 25 \geq T > 50 \\ 0.04(T - 50) - 0.9 & \text{if } 50 \geq T > 85 \end{cases}$$

$$V_{out}(V_s, T) = 0.8V_s * (1 + FS(T))$$

FS is the percentage of temperature influence, T is current temperature (relative to 0°C), and V_s is the output voltage dependent solely on the pressure.

- muRata NXFT15XV103FEAB050 temperature sensor [9]

This is a thermistor that was used as a temperature sensor. Comparing the measurements of the SCD41 temperature, this thermistor is capable of providing an update

change every 3s. Additionally, the temperature offset is less than 2% which complies with requirement #7. Since thermistors changes its resistance based on current temperature, the following equation describes the current resistance with reference of ambient temperature (in Kelvin)

$$R_T = R_1 e^{B(\frac{1}{T_2} - \frac{1}{T_1})}$$

Where R_T is the new resistance value of the thermistor, R_1 is the resistance value at $25[^\circ C]$; in this thermistor $T_1 = 10k[\Omega]$, $B = 3936[K]$ which is the thermistor constant, T_2 is the current temperature, and T_1 is the reference temperature ($25[^\circ C] = 298.15[K]$).

ii. Sensor Electrical Design

This section discusses about the use of electrical methodology to understand what are the current demands of the mentioned sensors, MCU selection, and other proper techniques.

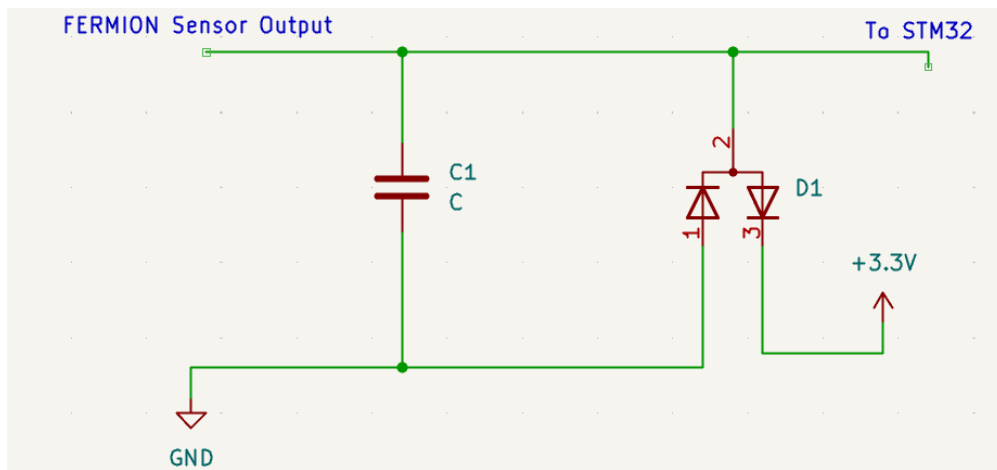
For the electrical integrations to the sensors to an MCU, the team needed to comprehend what kind of GPIO pins were necessary for utilizing the previous sensors mentioned. Since the project required a considerable amount of ADC pins inputs, DAC outputs, and PWM signals; the team decided to use a STM32 NUCLEO-64 STM32F446RE as the MCU for controlling every sensor, peltier signals, and motor drivers. However, this board is does not have the capability of video streaming. As a result, the team decided to include a Raspberry Pi 5 (8Gb) as their secondary MCU. This is further explored in the Embedded Systems section.

The demand for the sensor electrical design can be broken down in the following:

1. Signals going into the STM32 must be filtered.
2. If any signal needs amplification from the max GPIO the STM32 can provide, it must be done by using external hardware.
3. Any incoming signals that is not in the range of the STM32 must be filtered down, omitted, or transformed into something the MCU can handle.
4. Power rails must be considered unstable, this is to encourage better protection of the overall system.
5. Any modifications from the sensors must adhere (if possible) to the requirement #5 in the previous section.

Signals incoming to the STM32 must be filtered in order to aid the Embedded software side of the project, doing so facilitates the work of both parties in regards of signal handling. In these case, the first approach would be to create a decoupling capacitor to avoid high frequency ripple as this would comply with requirement #1.

In addition, if the sensors outputs something greater than +3.3V or something less than 0V would negatively impact the micro-controller ADC pins. Thus, a diode array has been placed on the same rail. This diode array would create a range in which a signal can be read through this pins. The anode side of the array is set to the minimum voltage level a node can be, where as the cathode is set to the maximum voltage level. This would comply with the requirement #3.



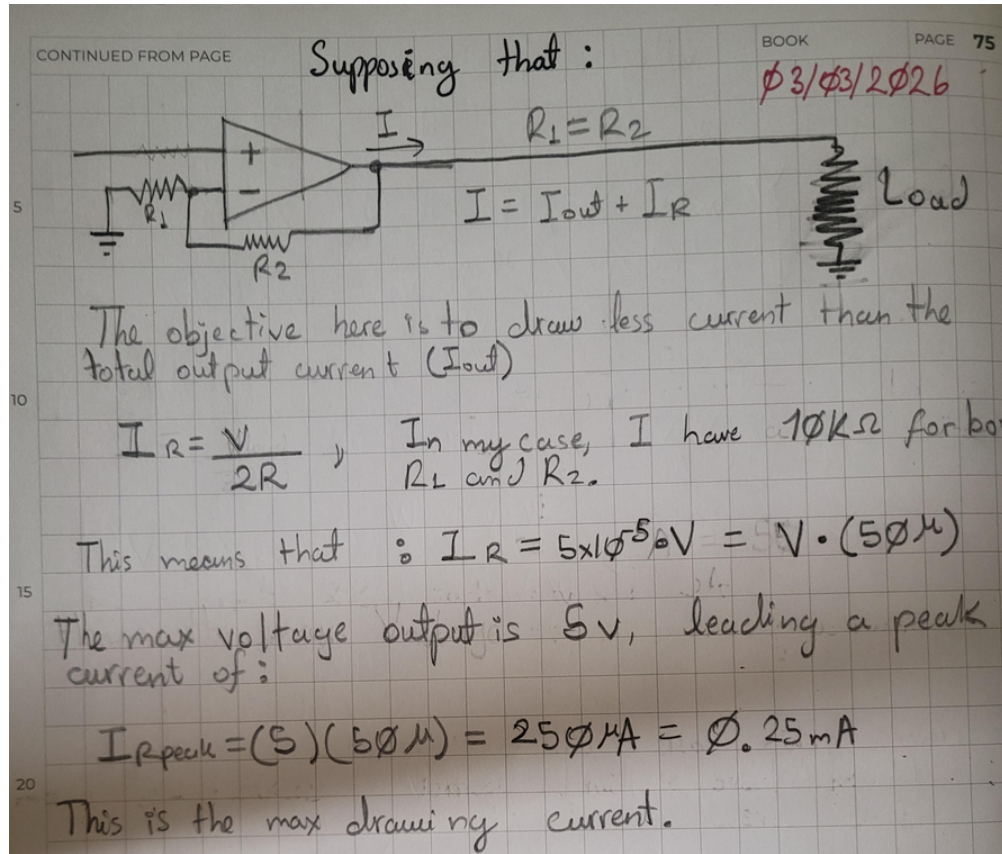
This set up shows how an array diode and decoupling capacitor can be set to work as one.

This image was made through the use of Kicad.

In regards of power filtering, the need for avoiding any sudden ripple across the power line is necessary for any chip in the electronic industry. Thus, the best approach was to use decoupling capacitors. Unlike the the decoupling capacitors shown above, power rails required to be sturdier towards ripples. Since the project is a Rover, it is expected a lot of vibrations would happen within the chassis. Thus, for power rails ceramic capacitors are not useful due that they tend to change their capacitance as vibrations occur [15]. Instead, the better option was to use Tantallum capacitors due to its capacitance is not affected as much for vibrations. A disadvantage of the Tantallum capacitor is they increase capacitance as temperature increases; but since the team is trying to filter high frequencies, this would not cause any problems for the project. Thus, requirement #4 is accomplished.

For this project, the team observed that many outputs of the STM32 would not have enough potential to interact with the motor drivers. As a result, the team needed to amplify this signals. The solution the team provided is to use an operational amplifier with a gain of

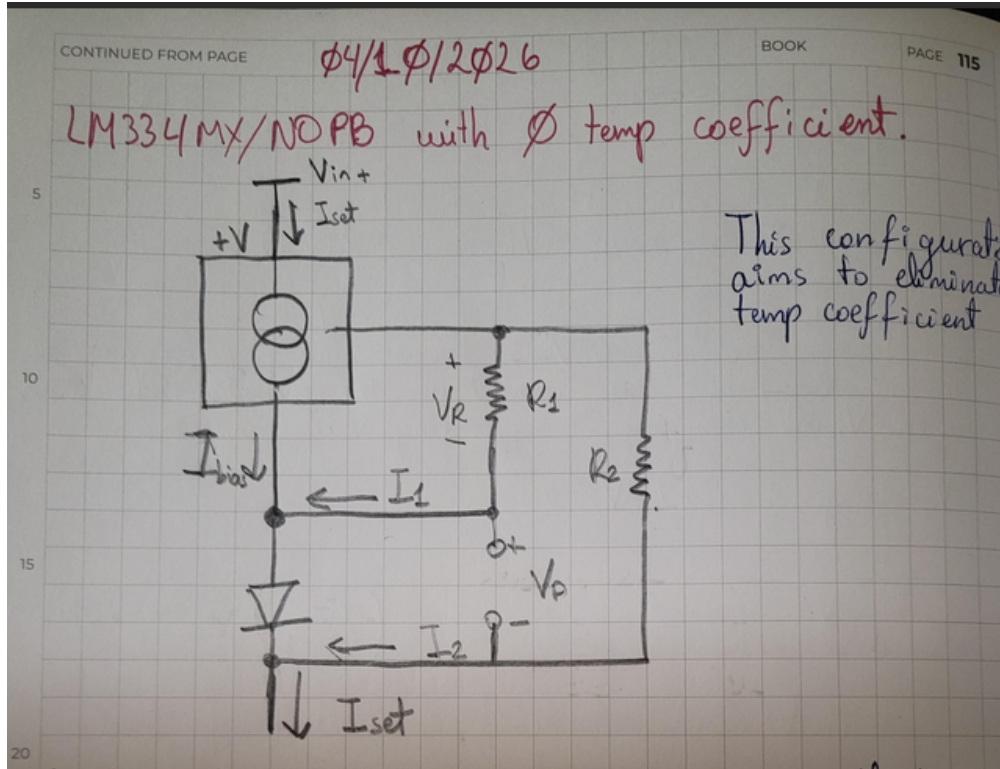
two. In this way, the team can send +3.3V to become +5V without the need of modifying the internal hardware of the MCU. Thus, this complies with requirement #2.



This image includes the set up for the GPIO pin amplification. Additionally, it determines how much current is drawn by the feedback loop.

Since there was a demand for external hardware attached to the STM32, a daughter board was created. In this paper, we refer it as the STM32 shield. This shield objective is to comply with all the requirements listed in this section.

As previously explained in the last section, the Barometer did not meet the requirement #5 of that section. Thus, the team decided to build a module based on the needs for the barometer. The first being a constant current source for an steady $100\mu A$ and amplifying the signal to be visible between the range of 0V to +3.3V. This led to the use of the LM334MX/NOPB [16]. The datasheet offers a great insight on how to set up the circuitry for a constant current sensor.



This configuration aims to eliminate temperature coefficient. I_{set} is defined as the sums of all the currents.

$$I_{set} = I_{bias} + I_1 + I_2 \quad I_1 = \frac{V_R}{R_1} \quad I_2 = \frac{V_R + V_D}{R_2}$$

I_{set} would be $100\mu A$ for the barometer [8].

Temp coefficient at $25^\circ C$ is $227\mu V/^\circ C$. Thus, minimizing the temperature coefficient at $25^\circ C$:

$$I_{set} = I_{bias} + I_1 + I_2$$

$$\frac{dI_{set}}{dt} = \frac{dI_{bias}}{dt} + \frac{dI_1}{dt} + \frac{dI_2}{dt}$$

$$\frac{dI_{set}}{dt} = 0 = \frac{227\mu V/^\circ C}{R_1} + \frac{227\mu V/^\circ C + D_T}{R_1}$$

D_T as the temperature coefficient of a diode. To facilitate the process, the team opted to use the same diode the datasheet recommended in their equations (and was easier to solder due that it was a through hole).

$$\frac{dI_{set}}{dt} = 0 = \frac{227\mu V/^\circ C}{R_1} + \frac{227\mu V/^\circ C - 2.5mV/^\circ C}{R_1}$$

$$\frac{R_2}{R_1} = \frac{227\mu V/^\circ C - 2.5mV/^\circ C}{227\mu V/^\circ C} \quad R_2 = 10R_1$$

Utilizing the graph (figure 9) in the datasheet, its concluded the voltage across R_1 is defined as $V_{R1} \approx 64mV$. This is useful as it describes the potential across R_1 and allows to omit I_{Bias}

$$\begin{aligned}
V_R &= 64mV(1 + 0.059) \approx 67.7mV \\
I_{set} &= 100\mu A = \frac{67.7mV}{R_1} + \frac{67.7mV+0.6V}{10R_1} \\
100\mu A &= \frac{(10)(67.7mV)+67.7mV+0.6V}{10R_1} \\
100\mu A &= \frac{0.134V}{R_1} \\
R_1 &= \frac{0.134V}{100\mu} = 1340\Omega \\
R_2 &= 10R_1 = 13.4k\Omega
\end{aligned}$$

Thus, the component selection for an steady current supply has been obtained. The next step is convert the output of the barometer into a single analog voltage out. The team's only idea for achieve this result was to use an instrumental op-amp.

The equation for a voltage divider can be expressed in two ways:

$$\begin{aligned}
V_{out} &= (V_2 - V_1) + V_{ref} \\
V_{out} &= G(V_{diff} + V_{ref})
\end{aligned}$$

In this case V_{out} is noted as the voltage output of the instrumental amplifier, V_1 and V_2 are defined as the differential input, and V_{ref} is voltage reference with (our application this is 0V). In regards of the differential input of the barometer, looking at the datasheet [8], it seems the maximum value is close to be $47.5mV$. Therefore, this maximum value must be equal (or at least close) to $+3.3V$. For this, it would be best to use the second equation.

$$\begin{aligned}
V_{out} &= G(V_{diff} + V_{ref}) \\
3.3V &= G(47.5mV) + 0V \\
G &= \frac{3.3V}{47.5mV} \approx 69.474 \Rightarrow 69.5
\end{aligned}$$

Understanding the needed gain for the barometer, the team decided to use an instrumental amplifier because it was the most affordable choice and easy to implement. Based on our suppliers catalog, the instrumental amplifier chosen was INA126 [17]. This op-amp datasheet provide an easy equation for deciding what is the resistance gain R_G

$$\begin{aligned}
G &= 5 + \frac{80k\Omega}{R_G} \\
R_G &= \frac{80k\Omega}{G-5} = \frac{80k\Omega}{69.5-5} \\
R_G &= 1240\Omega
\end{aligned}$$

With a more clear idea of what resistance value the gain of the instrumental amplifier must be, the easiest option was to obtain a potentiometer capable of going more than R_G . Thus, this concludes the design of the barometer module.

e. Embedded Systems

i. Architecture

The embedded-system architecture was designed around a split-control model. A Raspberry Pi handles high-level computing tasks, while an STM32 handles deterministic low-level control. This split was selected because the ROV requires both operator-facing functions and real-time embedded behavior. The Raspberry Pi is well suited for dashboard hosting, camera handling, Ethernet communication, telemetry parsing, and logging. The STM32 is better suited for motor PWM generation, direction control, sensor acquisition, packet validation, and fail-safe behavior.

A Raspberry Pi-only design would simplify the number of processors, but it would place motor control under a Linux operating system that is not deterministic. A microcontroller-only design would provide predictable timing, but it would make camera streaming, dashboard hosting, network communication, and logging significantly more difficult. The final Pi/STM32 split therefore balances usability, processing capability, timing reliability, and implementation complexity.

The STM32 receives motor commands from the Raspberry Pi and converts them into motor-driver outputs. It also collects sensor data from analog and digital sensors and returns telemetry to the Raspberry Pi. This keeps the STM32 responsible for the robot-facing control loop and keeps the Raspberry Pi responsible for the operator-facing interface.

ii. Communication

UART was chosen for communication between the Raspberry Pi and STM32 because the two processors are physically close inside the robot, the required data rate is low, and the protocol is simple to debug. The Pi-to-STM32 command packet is fixed-length and deterministic. This reduces parsing complexity and allows corrupted commands to be rejected before they affect motor outputs.

The motor command packet uses the following format:

Table 2: Pi-to-STM32 Motor Command Packet

Byte	Field	Description
0	Header	Fixed value 0xAA
1	Left speed	8-bit PWM command, 0–255
2	Left direction	Left motor direction bit
3	Right speed	8-bit PWM command, 0–255
4	Right direction	Right motor direction bit
5	Checksum	Sum of bytes 1–4 modulo 256

The fixed header identifies the start of a valid packet. The payload carries left and right speed and direction values. The checksum allows the STM32 to reject corrupted packets. This structure was selected because the command requirements are simple: the STM32 only needs enough information to update two motor speeds and two motor directions.

Telemetry is sent in the reverse direction, from the STM32 to the Raspberry Pi. The STM32 packages sensor values into a readable telemetry frame and transmits them to the Pi for dashboard display. The general telemetry format is:

$$S:CO_2,CO,H_2S,CH_4,TEMP,BARO,HUMID. \quad (7)$$

An ASCII telemetry format is less compact than a binary format, but it is easier to inspect during testing. Since the telemetry rate is approximately 5 Hz, the bandwidth penalty is acceptable for the prototype.

iii. Safety

The embedded safety design focuses on preventing invalid or stale commands from controlling the motors. The STM32 checks each received motor packet before updating motor outputs. If the packet header is incorrect or the checksum does not match, the packet is discarded and the previous valid command is not replaced by corrupted data.

Timeout behavior is also required for safe operation. If the STM32 does not receive a valid command within a specified time window, the motor outputs should be forced to zero. This prevents the ROV from continuing to drive if communication is lost between the operator station, Raspberry Pi, and STM32. This behavior is especially important because the ROV is intended for confined underground spaces where uncontrolled motion could damage the vehicle or surrounding environment.

This safety model keeps the final authority over motor outputs on the STM32. Even if

the dashboard, camera stream, or Raspberry Pi software slows down or fails, the STM32 can still enforce motor-output behavior locally.

6. Subsystem Integration

a. Data Flow

The data flow begins at the ROV sensor suite and ends at the operator dashboard. Analog gas sensor outputs are routed to STM32 ADC inputs. Digital environmental sensors communicate with the STM32 through serial peripheral buses. The STM32 collects these values, formats them into telemetry frames, and sends them to the Raspberry Pi. The Raspberry Pi parses the telemetry and updates the dashboard so the operator can view live environmental data.

The telemetry path can be summarized as:

$$\text{Sensors} \rightarrow \text{STM32} \rightarrow \text{Raspberry Pi} \rightarrow \text{Dashboard} \rightarrow \text{Operator.} \quad (8)$$

This data path is central to the purpose of the ROV. The robot is not only a mobile platform; it is a remote monitoring system. The operator must be able to view gas, temperature, humidity, pressure, and system-state information without entering the hazardous environment.

b. Control Flow

The control flow moves in the opposite direction. The operator sends keyboard commands from the control computer. These commands travel over the CAT6a tether to the Raspberry Pi. The Raspberry Pi converts the operator command into a fixed UART packet and sends it to the STM32. The STM32 validates the packet and updates the motor PWM and direction outputs.

The control path can be summarized as:

$$\text{Operator} \rightarrow \text{Raspberry Pi} \rightarrow \text{STM32} \rightarrow \text{Motor Driver} \rightarrow \text{Motor.} \quad (9)$$

This structure separates the long-distance communication link from the local embedded-control link. Ethernet over CAT6a is used for the operator connection because it supports long-distance communication, dashboard access, and camera streaming. UART is used locally because it is simple, deterministic, and sufficient for motor commands and telemetry between the two processors.

i. Industry Standard DB25 Connector

The DB25 connector supports subsystem integration by consolidating wiring between the electrical system, sensor package, and embedded controller. This improves serviceability because sensors and external harnesses can be connected through a defined interface instead of loose individual wires. It also helps future teams troubleshoot the system by giving the wiring harness a repeatable physical structure.

From the embedded-system perspective, the DB25 connector matters because reliable sensor and control wiring is required before firmware validation can be meaningful. If the connector pinout, protection circuitry, and signal routing are consistent, then software errors can be separated from wiring faults during testing.

c. Power Distribution

The power distribution subsystem connects the battery, motor drivers, cooling hardware, compute hardware, embedded controller, and sensors into one electrical system. This section is part of subsystem integration because each major subsystem depends on the power architecture being stable, correctly routed, and separated by load type.

The rover uses a 25.6 V nominal LiFePO_4 battery source. This battery source feeds the motor drivers and the buck-regulated PDB. The PDB then generates the lower-voltage subsystem rails. The 12 V, 8 A rail supports the Peltier cooling modules and heatsink fan loads. The 5 V, 5 A rail supports the Raspberry Pi 5. The 5 V, 1 A rail supports the STM32. A 3.3 V, 250 mA LDO supports the sensor loads.

This separation is important because the rover contains both high-current loads and noise-sensitive electronics. Motors and Peltier cooling modules can create larger load changes and switching transients, while the Raspberry Pi, STM32, and sensors require stable low-voltage rails. Keeping these rails separated reduces the chance that motor current spikes or Peltier switching disturb sensor measurements, reset the compute hardware, or interfere with embedded control.

The power distribution system also supports embedded control of the cooling subsystem. The STM32 provides PWM control signals for the Peltier MOSFET switching path, but the PDB supplies the actual current required by the Peltier modules. This means the embedded system controls the cooling behavior, while the power system provides the regulated energy path needed to drive the cooling load.

Full vehicle validation still required the rails to be tested together with motors, cooling, Raspberry Pi, STM32, and sensors connected at the same time. Therefore, the power distribution design should be treated as integrated at the architecture and board-design level,

but only partially validated at the full-system load level.

d. Key Integration Challenges

The largest integration challenge was that embedded testing depended on the readiness of the mechanical, power, and electrical subsystems. The command and telemetry pipeline could be validated on the bench, but full vehicle validation required the drivetrain, chassis, icebox, power distribution, sensor board, and wiring harness to be physically complete.

Thermal constraints also affected integration. The ROV is intended to operate in a high-temperature environment, so the Raspberry Pi, STM32, sensors, motor drivers, voltage regulators, and wiring must all remain within safe operating conditions. Embedded software can monitor and control parts of this system, but final thermal validation requires the completed vehicle.

PCB design and component sizing also created integration dependencies. Sensor output ranges, ADC input protection, I2C level shifting, power-rail capacity, and connector pin assignments had to be compatible with the STM32 and Raspberry Pi interfaces. These dependencies made incremental bring-up necessary.

7. Development Methodology

a. Mechanical

i. CAD to Prototype to Iteration

CAD development for the chassis and drivetrain initially began with arbitrary dimensions, but defined in such a way that the length, height, and width of the chassis could be changed with the modification of one or two variables within Onshape. In doing so, the shape of the ROV could be determined before any component decisions were made: once the electrical, heat management, and drivetrain subsystem components were selected, chassis dimensions could be shrunk to fit. The most pertinent constraint on the chassis size ended up being that of the icebox, which was made as small as possible to reduce surface area and thus efficiency losses. During development, multiple prototype chassis and iceboxes were constructed for the purposes of iteration, as overlooked design considerations and flaws were much easily pointed out by team members with a physical model to compare and use. After each prototype was fabricated and introduced to the team, iteration on the CAD design would begin based on feedback, with a new prototype fabricated and introduced to team members for the purposes of further iteration. This process combined with Onshape’s collaborative features allowing multiple people to work on the same design, and proved invaluable for integration.

b. Electrical

i. Schematic to Simulation to PCB Design

The electrical development work included two main printed circuit board (PCB) efforts: the PDB and the STM32 shield. The PDB implemented the rover’s regulated power rails and Peltier switching paths, while the STM32 shield provided a repeatable interface between the microcontroller, sensors, protection circuitry, and external wiring harnesses. Together, these boards turned the system architecture into physical electrical hardware.

The PDB was developed to convert the battery source into the subsystem rails required by the rover. The PDB accepts the 25.6 V nominal battery source and generates regulated outputs for the cooling, compute, embedded-control, and sensor subsystems. The main rails included 12 V for Peltier cooling and fan loads, 5 V for the Raspberry Pi 5, 5 V for the STM32 path, and a 3.3 V LDO path for sensors. The board also included MOSFET switching stages so the STM32 could control Peltier average power using PWM.

The PDB design progressed from power architecture modeling to regulator selection, schematic capture, board layout, fabrication preparation, and bench testing. Texas Instruments WEBENCH Power Designer and LTpowerCAD were used during the regulator-selection and power-architecture stages, while KiCad was used for schematic capture and PCB layout. After design-review feedback, the regulator design was revised to show more engineering intent rather than relying only on tool-generated values. The 12 V regulator stage was reviewed against datasheet equations, including switching-frequency tradeoffs, inductor ripple-current targets, load-step assumptions, input and output capacitance, feedback resistor values, and tolerance effects. The selected switching frequency was reduced to 500 kHz to better balance efficiency, component size, cost, and electromagnetic-interference concerns.

The Peltier switching stage was developed as a low-side N-channel MOSFET circuit. A relay was considered first because it was simple for on/off switching, but a MOSFET was selected because the cooling system required repeated electronic switching and PWM control from the STM32. The first-pass gate network used a 470 Ω series gate resistor and a 47 k Ω gate pull-down resistor. These values were selected from STM32 General-Purpose Input/Output (GPIO) drive limits and from the need to keep the MOSFET off when the controller pin is reset or high impedance. The IRLML6244 was selected as the first MOSFET candidate because its low-gate-voltage $R_{DS(on)}$ reduced estimated conduction loss for the measured Peltier current. The AO3400A was kept as a backup candidate because it provided higher voltage margin and lighter gate-drive requirements.

The PDB layout was completed in KiCad. Regulator placement, high-current routing,

ground returns, and MOSFET driver placement were reviewed before fabrication. Design Rule Check (DRC) errors were cleared, Gerber files were generated, and the board files were checked for manufacturability. A MOSFET symbol-to-footprint mismatch was later found before fabrication and corrected. This prevented the gate, source, and drain connections from being fabricated incorrectly.

In parallel, the STM32 shield was developed to make sensor and control integration more repeatable. The shield routed sensor outputs to STM32 pins, consolidated external wiring through a DB25 connector, and added protection circuitry for the microcontroller inputs. This was important because the rover used a mixture of analog sensor signals, digital communication buses, 5 V interfaces, and 3.3 V STM32 logic. Without a defined shield and connector interface, debugging would have depended on loose wiring and would have made it harder to distinguish software faults from wiring or protection-circuit faults.

The STM32 shield design focused on protecting the microcontroller and conditioning sensor signals. Voltage dividers, diode arrays, operational amplifiers, bypass capacitors, and level-shifting circuitry were used where needed to keep sensor and communication signals within the STM32-safe voltage range. The DB25 connector also helped standardize the wiring harness so that sensors and external subsystems could connect through a defined physical interface rather than through loose point-to-point wiring.

Board validation was also separated by board function. For the PDB, regulator simulation was limited because the regulator integrated-circuit models were not available or practical in the PSpice workflow. The MOSFET Peltier driver was simulated instead using a 12 V rail, an equivalent Peltier load resistance, the IRLML6244 MOSFET model, a 470 Ω gate resistor, a 47 k Ω pull-down resistor, and a 0 V to 3.3 V PWM source representing the STM32 output. The simulation temperature was set to 50°C to match the rover’s target environment. The simulation showed the expected low-side switching behavior.

For the STM32 shield, validation focused on rail isolation, signal conditioning, level shifting, and input protection. Power-rail checks identified and corrected a solder bridge on a small surface-mount component. Operational amplifier testing confirmed the intended gain behavior. Level-shifter testing checked I2C signal behavior across frequency. Sensor-output bus testing used varying input voltages to verify that the diode-array protection clamped voltages within the STM32-safe range. These tests showed that the shield could support controlled sensor-interface bring-up, although final gas-sensor calibration remained separate from board-level electrical validation.

Together, the PDB and STM32 shield reduced system-integration risk. The PDB organized the rover’s power conversion and high-current switching, while the STM32 shield organized the sensor, protection, and wiring interface. The remaining limitation was that

both boards still required full vehicle-level validation with motors, cooling, sensors, Raspberry Pi, STM32, and dashboard operation connected together.

c. Embedded

i. Incremental Bring-Up

The embedded subsystem was developed using incremental bring-up. This means each hardware and software interface was tested separately before being combined into the full system. This approach was used because the ROV contains several interacting subsystems, and testing the entire system at once would make failures difficult to isolate.

The embedded bring-up sequence was:

1. Configure STM32 GPIO, timers, UART, ADC, DMA, and I2C.
2. Verify STM32 PWM outputs for motor control.
3. Verify STM32 direction GPIO outputs.
4. Verify Raspberry Pi to STM32 UART packet transmission.
5. Verify STM32 packet validation and checksum rejection.
6. Verify ADC acquisition for analog sensor inputs.
7. Verify digital sensor polling.
8. Verify STM32 telemetry frame generation.
9. Verify Raspberry Pi telemetry parsing.
10. Verify dashboard display of live sensor and system values.
11. Verify operator input to motor-output behavior.

This sequence reduced integration risk because each layer had a known test result before the next layer was added.

ii. System Bring-Up

System bring-up followed a subsystem-to-integration-to-full-pipeline approach. First, the STM32 was tested as a standalone controller. Next, the Raspberry Pi was connected to the STM32 over UART. Then the dashboard was connected to the Raspberry Pi over the tethered network. Finally, operator commands and telemetry were tested as a complete loop.

The intended full embedded pipeline was:

$$\text{Operator} \rightarrow \text{Pi} \rightarrow \text{STM32} \rightarrow \text{Motor Output} \quad (10)$$

and

$$\text{STM32 Sensor Data} \rightarrow \text{Pi} \rightarrow \text{Dashboard}. \quad (11)$$

This pipeline was successfully demonstrated on the bench. However, full mounted vehicle validation required final mechanical, thermal, and power integration, which was not completed before the final design defense.

8. Validation Methodology

a. Communication Validation

i. Packet Integrity

Communication validation first tested packet integrity between the Raspberry Pi and STM32. The Pi generated fixed motor packets containing a header, left speed, left direction, right speed, right direction, and checksum. The STM32 checked the packet before updating motor outputs.

A valid packet was required to satisfy the following conditions:

1. The packet must have the correct fixed length.
2. The first byte must match the fixed header value.
3. The received checksum must match the calculated checksum.

If any of these checks failed, the STM32 rejected the packet. This confirmed that invalid packets would not directly update motor outputs.

ii. Latency and Packet Loss

The communication target was less than 150 ms round-trip latency and less than 2% packet loss over the tethered communication path. Bench testing showed that the command and telemetry path was within the target range during prototype validation. Final measured values should be inserted from the team validation log if available.

The key result is that the communication path was sufficient for bench-level teleoperation. The operator could send commands, the STM32 could respond, and telemetry could return to the dashboard at approximately 5 Hz.

b. Control Validation

i. Motor Response

Control validation tested whether operator commands produced the expected motor output behavior. The test path was:

$$\text{Operator Input} \rightarrow \text{Raspberry Pi} \rightarrow \text{STM32} \rightarrow \text{Motor Driver.} \quad (12)$$

The command path passed at the bench level because motor outputs changed in response to operator commands. Left and right commands could be generated independently, allowing differential-drive behavior such as forward motion, reverse motion, turning, and stopping.

ii. Fail-Safe Behavior

Fail-safe validation tested whether unsafe command conditions were rejected. Invalid packets did not update the motor outputs. This confirms that the STM32 packet-validation layer worked as intended.

The remaining fail-safe behavior that should be verified before field deployment is stale-command timeout braking. In this test, the operator command stream should be interrupted while the STM32 is active. The passing condition is that both motor outputs return to zero within the required timeout window.

c. Sensor Validation

i. Compare Readings to Expected Values

Sensor validation was separated into acquisition validation and calibration validation. Acquisition validation checks whether the STM32 can read sensor signals and transmit them to

the dashboard. Calibration validation checks whether the readings are accurate compared to known references.

The acquisition path was prepared for gas and environmental measurements. Analog gas sensors route through STM32 ADC inputs, while digital environmental sensors are polled and included in telemetry. The Raspberry Pi receives these telemetry frames and displays the values on the dashboard.

ii. Accuracy Targets

The target accuracy for relevant gas sensors was less than 10% error compared to reference measurements. The target for environmental sensors was tighter, with temperature, pressure, and humidity compared against calibrated reference instruments.

Final gas-sensor calibration was not completed. Therefore, the sensor subsystem should be described as acquisition-ready rather than fully accuracy-validated. The system can collect and display sensor readings, but the readings should not be treated as safety-certified gas measurements until reference calibration is completed.

d. Power Validation

Power validation was separated into subsystem-level validation and full-load system validation. This distinction was necessary because several individual power-system tests could be completed before the full rover was mechanically and electrically integrated. The purpose of the subsystem-level tests was to reduce risk before connecting sensitive components such as the Raspberry Pi, STM32, sensors, motors, and Peltier modules together.

i. Subsystem-Level Power Tests

The subsystem-level power tests focused on proving individual loads, switching behavior, and rail behavior before the full rover was connected together. These tests included Raspberry Pi power draw, Peltier load sizing, no-load motor behavior, MOSFET switching simulation, PWM behavior, and an individual 5 V rail no-load check.

Table 3: Power Subsystem Validation Summary

Validation Item	Evidence	Status
Raspberry Pi 5 power draw	Tested under multiple operating states, including camera-related operating conditions	Passed
Peltier load sizing	Current draw measured during bench voltage-sweep testing	Passed
Motor power behavior	Motor and driver tested under no-load command levels	Partial
MOSFET switching	IRLML6244 simulated at 50°C with a Peltier-equivalent load	Passed
PWM behavior	Duty-cycle and frequency behavior checked in simulation	Passed
5 V, 5 A rail output	Individual no-load rail check measured 5.008 V	Partial
Full rail stability	12 V, 5 V, and 3.3 V rails tested under motors, cooling, Raspberry Pi, STM32, and sensor load	Incomplete

The Raspberry Pi power test showed that the compute subsystem could be powered under multiple operating states. The Peltier voltage-sweep test gave project-specific current data for the thermal load instead of relying only on datasheet assumptions. The MOSFET simulation showed that the selected low-side switching topology behaved as expected under a simplified Peltier-equivalent load.

The 5 V, 5 A rail no-load test measured 5.008 V, which confirmed basic regulation for that rail. However, this result did not prove full-load stability. A no-load voltage check only shows that the regulator can reach the target voltage without the final subsystem load connected.

ii. Remaining Full-Load Validation

Full power validation still required the rails to be tested with motors, cooling, Raspberry Pi, STM32, and sensors connected together. This full-load test was necessary because motors and Peltier modules can create larger load changes than the compute and sensor electronics. These load changes could cause rail sag, reset the Raspberry Pi or STM32, or introduce noise into the sensor subsystem.

The power subsystem should therefore be described as partially validated. Subsystem-level tests reduced design risk, but full-load rail stability and full-system runtime validation remained incomplete before the final design defense.

e. System Validation

i. End-to-End Pipeline

The end-to-end embedded pipeline was validated at the bench level. The operator could send commands to the Raspberry Pi, the Raspberry Pi could send packets to the STM32, the STM32 could update motor outputs, and telemetry could return to the dashboard.

The feedback loop can be summarized as:

$$\text{Operator} \rightarrow \text{Robot} \rightarrow \text{Feedback Loop.} \quad (13)$$

This result proves that the critical command and data paths worked. However, it does not prove that the complete ROV is field-ready because final mounted vehicle validation was not completed.

ii. Runtime and Thermal Testing

Runtime and thermal testing remained incomplete at the full vehicle level. The embedded subsystem can support these tests by logging telemetry, internal temperature, command status, and communication behavior. However, proving the full runtime and thermal requirements requires the completed chassis, icebox, power system, drivetrain, and electronics package.

The final required test should operate the complete ROV under the target thermal condition while monitoring communication, sensor telemetry, power stability, and motor behavior. Until that test is completed, the system should be described as partially validated.

9. Analytical Methodology

The analytical methodology combined requirement decomposition, first-principles calculations, spreadsheet modeling, design-tool output, simulation, bench measurements, and subsystem-level validation. The purpose of this methodology was to move the project from broad system goals toward traceable engineering decisions that could be checked, revised, and validated.

The team began by decomposing the ROCK system into subsystem requirements. The rover had to move through a confined environment, protect electronics from elevated tem-

perature, collect environmental data, communicate with an operator station, and operate from an onboard power source. Each subsystem lead then translated those system goals into lower-level design questions. Mechanical analysis focused on drivetrain geometry, chassis packaging, ground clearance, and shock absorption. Thermal analysis focused on heat-sensitive electronics, Peltier cooling, insulation, airflow, and icebox packaging. Electrical and power analysis focused on sensors, protection circuitry, voltage rails, battery capacity, regulator sizing, and wiring. Embedded analysis focused on command packets, telemetry formatting, motor control, and fail-safe behavior.

For the drivetrain, the team used analytical models to estimate the force and torque required to move the rover. The motor torque requirement was calculated from incline force, rolling resistance, acceleration force, wheel radius, number of motors, and drivetrain efficiency. The simplified tractive-force model was

$$F_{\text{required}} = mg \sin(\theta) + C_{rr}mg \cos(\theta) + ma, \quad (14)$$

where m is rover mass, g is gravitational acceleration, θ is incline angle, C_{rr} is the rolling-resistance coefficient, and a is acceleration. This force was converted into a per-motor torque requirement using

$$T_{\text{motor}} = \frac{F_{\text{required}}r}{N_{\text{motors}}\eta_{\text{mech}}}, \quad (15)$$

where r is wheel radius, N_{motors} is the number of driven motors, and η_{mech} is drivetrain efficiency. This calculation produced a required torque of 1.48 N·m per motor. A 24 V, 60 W brushless motor rated at 3.8 N·m was selected because it provided torque margin above the calculated requirement.

The mechanical subsystem also used energy-based reasoning for the shock absorber selection. The team compared different shock absorber lengths and treated the suspension problem as an energy absorption problem, where the suspension should absorb impact energy from drops or obstacles. This analysis helped justify the larger shock absorber option because it provided more travel distance and better obstacle recovery, even though it required the rover to operate at lower speed over larger depressions.

For the capstan drivetrain, the team used the capstan relationship to reason about how rope wrap angle affects the required holding force. The capstan equation was used as

$$T_{\text{hold}} = \frac{T_{\text{load}}}{e^{\mu\theta}}, \quad (16)$$

where T_{hold} is the holding force, T_{load} is the load force, μ is the friction coefficient, and θ is the rope wrap angle. This analysis supported the design decision to increase wrap

engagement and improve the capstan drum geometry when rope slipping became a drivetrain limitation during testing.

For the thermal subsystem, the team used a combination of heat-management reasoning, component fit checks, and bench testing. The icebox design was evaluated around the need to fit heat-sensitive electronics, reduce insulation gaps, maximize insulation thickness within the packaging constraints, and maintain a temperature difference between the inside electronics volume and the external environment. Peltier testing was used to compare the expected thermal behavior against project-specific measurements. These tests helped the team evaluate Peltier voltage, current draw, heatsink behavior, fan direction, and condensation effects.

For the power subsystem, the team used a power budget to translate subsystem choices into electrical requirements. Motors and Peltier modules dominated the system power demand, while the Raspberry Pi, STM32, fans, and sensors defined the lower-power rails. The power budget was revised as the design changed, including updates for additional Peltier modules, heatsink fans, motor-current assumptions, and state-based mission-time estimates. Where possible, measured current data was used to replace early datasheet assumptions.

Battery sizing used the estimated mission energy from the power budget. For a 1-hour mission estimate of 56.141 Wh and a 25.6 V nominal battery source, the required capacity was calculated as

$$C_{\text{required}} = \frac{56.141 \text{ Wh}}{25.6 \text{ V}} = 2.19 \text{ Ah.} \quad (17)$$

For a 1-hour mission, this corresponds to an average battery current of about 2.19 A. This calculation was used as a runtime estimate, not as a substitute for a completed full-system runtime test.

The regulator analysis used buck-converter design equations and design-tool outputs to review selected component values. The Power Distribution Board regulator stages were checked for switching-frequency tradeoffs, inductor ripple-current targets, input capacitance, output capacitance, load-step response, feedback resistor values, and tolerance effects. This review was important because the design originally relied heavily on design-tool output. The analytical review made the regulator choices more defensible by connecting the selected values to datasheet equations, load assumptions, and measured subsystem behavior.

The electrical subsystem used interface analysis to protect the STM32 and standardize sensor wiring. Sensor output ranges, analog input limits, 5 V and 3.3 V compatibility, level shifting, diode clamping, operational-amplifier gain, and connector pin assignments were reviewed so that sensor signals could be routed safely into the STM32 shield. The DB25 connector and STM32 shield were analyzed as integration tools because they reduced loose

wiring and made sensor, power, and protection connections more repeatable.

The embedded subsystem used protocol and timing analysis to separate operator-facing tasks from real-time control tasks. The Raspberry Pi handled dashboard, camera, Ethernet, telemetry parsing, and logging, while the STM32 handled motor timing, packet validation, sensor acquisition, and safety behavior. A fixed-length packet structure was used so that motor commands could be parsed predictably and rejected if the header or checksum was invalid. Telemetry formatting was kept readable during testing because the approximately 5 Hz telemetry rate made the bandwidth tradeoff acceptable for the prototype.

Simulation was used where practical. Regulator simulation was limited because the regulator integrated-circuit models were not available or practical in the selected PSpice workflow. The Peltier MOSFET driver was simulated instead using a simplified Peltier-equivalent resistive load. The simulation checked gate voltage, drain behavior, load current, duty-cycle response, frequency response, and load-resistance sensitivity at 50°C. This gave useful subsystem evidence, but it did not replace bench validation on the assembled board.

Bench testing was used to replace assumptions with project-specific data. The Peltier modules were tested with a voltage sweep to measure current and temperature behavior under the team’s heatsink and fan setup. The motors were tested under no-load command levels to verify basic motor-driver response and current draw. The Raspberry Pi was tested under multiple operating states to estimate compute-load demand. The STM32 shield was tested for rail isolation, operational-amplifier behavior, level-shifter behavior, and input clamping. The 5 V, 5 A rail was checked with no load and measured 5.008 V.

The validation methodology also changed as the team learned from testing. After the first Power Distribution Board fault, the rail-test method was revised to require visual inspection, unpowered continuity checks, resistance checks, diode-mode checks, no-load voltage testing, light-load testing, step-load testing, and temperature checks. This method prevents downstream rails from being tested before upstream rails are verified. It also makes each measurement repeatable by recording the rail, reference designator, probe points, input source, expected result, measured result, load, current limit, temperature, and follow-up action.

Overall, the analytical methodology connected subsystem requirements to calculations, tool outputs, simulations, prototypes, and bench evidence. The main limitation was that several results remained subsystem-level results. Full analytical closure still required final vehicle-level testing with the drivetrain, chassis, icebox, power distribution, sensors, Raspberry Pi, STM32, dashboard, and communication link integrated together.

10. Conclusion

The ROCK system is structured, justified, and partially validated. The final architecture separates operator-facing tasks from real-time vehicle control by using a Raspberry Pi for high-level processing and an STM32 for deterministic embedded behavior. This split supports the project requirements because the operator needs cameras, dashboard access, telemetry, and network communication, while the robot needs predictable motor control, sensor acquisition, packet validation, and safety behavior.

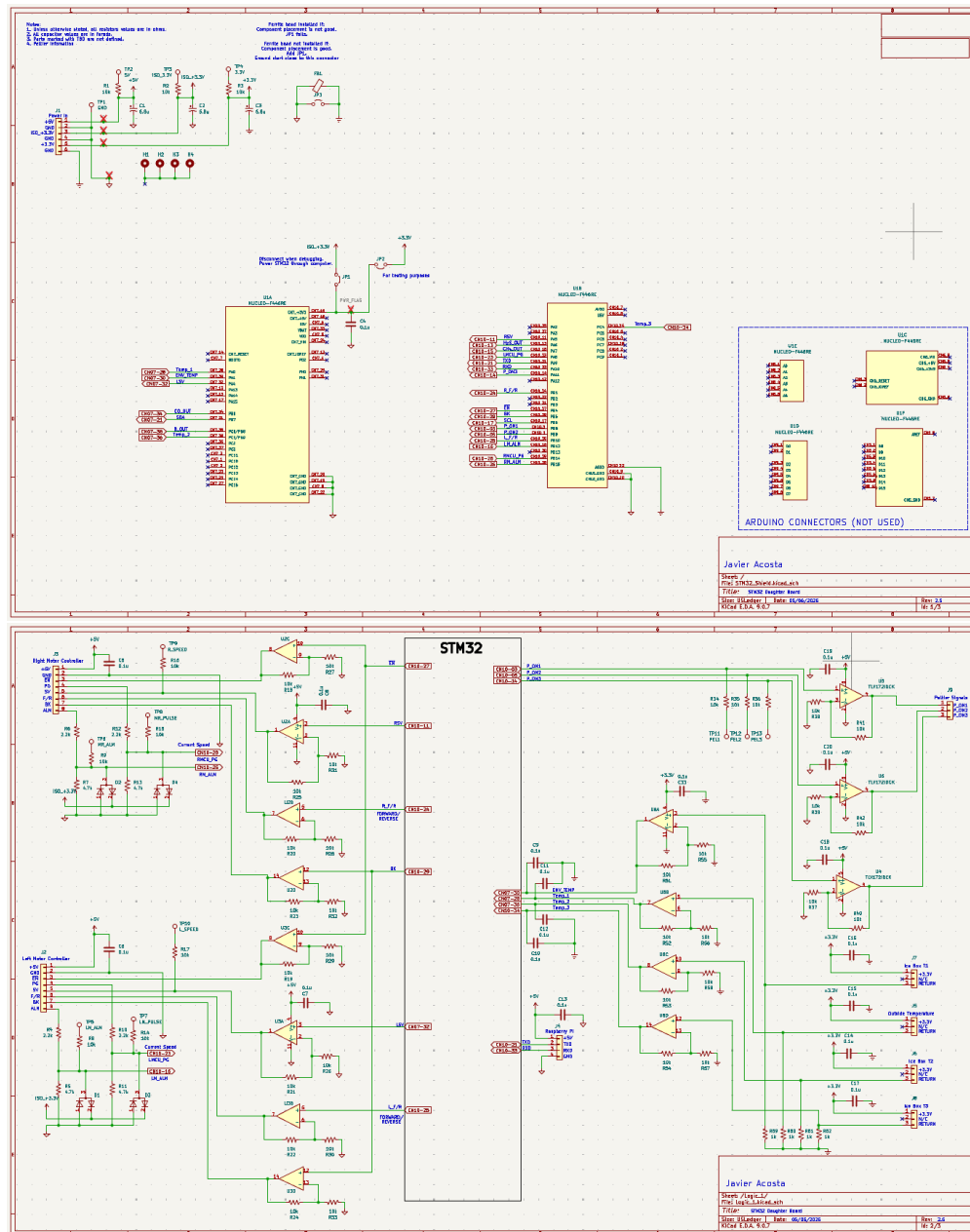
The embedded subsystem reached bench-level functionality. Operator commands were successfully passed from the control computer to the Raspberry Pi, converted into UART packets, received by the STM32, validated, and used to update motor outputs. Sensor and system telemetry were returned from the STM32 to the Raspberry Pi and displayed on the dashboard. Invalid packets were rejected rather than being allowed to update the motors. These results show that the critical command path, telemetry path, and basic safety path were functional.

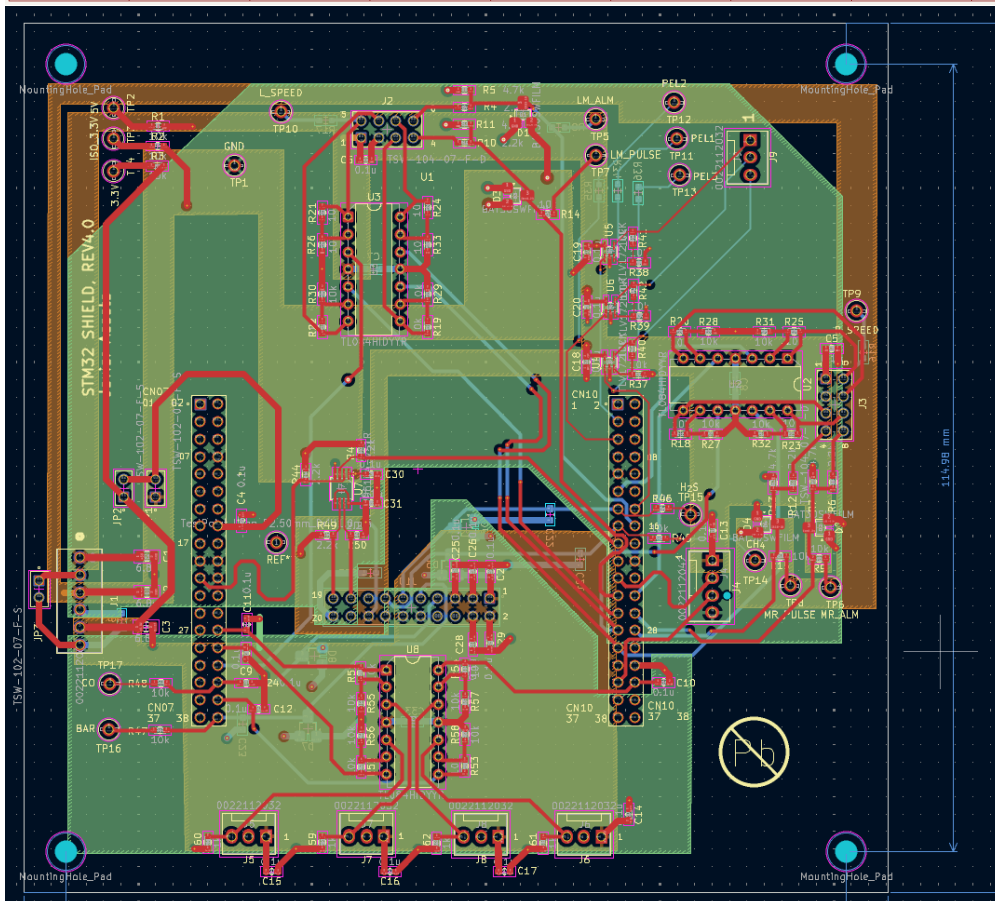
The system was not fully validated for deployment. The main unresolved requirements are full thermal validation, full runtime validation, calibrated gas-sensor validation, and field testing. The ROV still requires a complete mounted test with drivetrain, chassis, icebox, power distribution, sensors, cameras, dashboard, and embedded control operating together. The final result should therefore be described as a bench-validated prototype architecture rather than a field-ready robot.

Remaining work should focus on completing physical integration and proving the system under realistic conditions. The next team should run a full-load power test, a 50°C thermal chamber test, a calibrated gas-sensor comparison, a communication-loss braking test, and a field-style mobility test with live camera and telemetry feedback. Once these tests are completed, the ROCK platform would be much closer to a deployable low-cost tool for safer underground data collection.

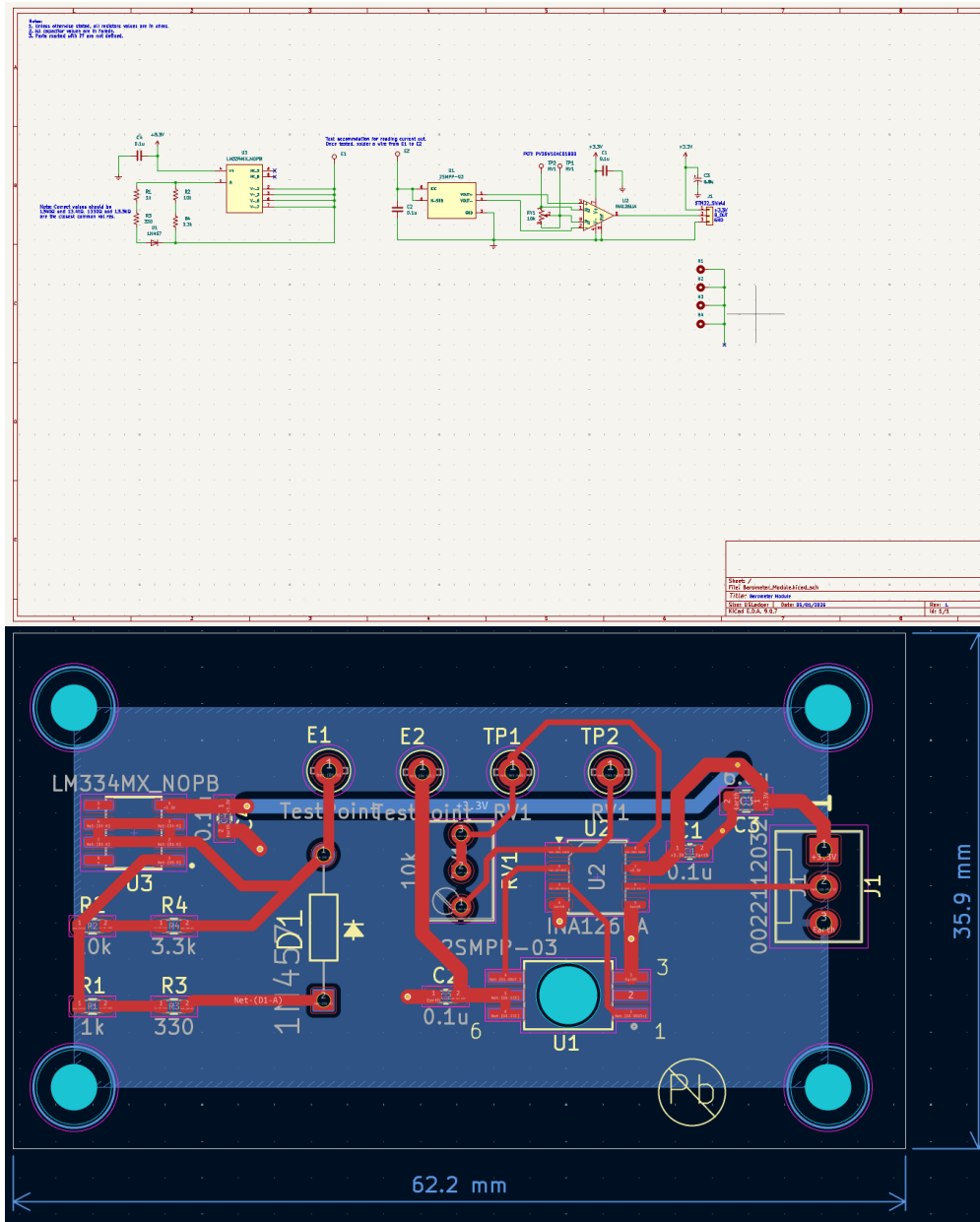
11. Schematics and PCB designs

a. STM32 Shield and PCB





b. Barometer Module and PCB



References

- [1] Zhang, Lianjie, et al. “Development of a Cleaning ROV for High-Temperature Radioactive Environment.” *Intelligent Robotics and Applications*, edited by Xuguang Lan et al., Springer Nature, 2025, pp. 116–30. Springer Link, https://doi.org/10.1007/978-981-96-0792-1_10.
- [2] M. T. Ohradzansky et al., ”Multi-Agent Autonomy: Advancements and Challenges in Subterranean Exploration,” in *Field Robotics*, vol. 2, pp. 1068-1104, June 2022, doi: 10.55417/fr.2022035.
- [3] “NUCLEO-F446RE - STM32 Nucleo-64 development board with STM32F446RE MCU, supports Arduino and ST morpho connectivity - STMicroelectronics,” www.st.com. <https://www.st.com/en/evaluation-tools/nucleo-f446re.html>
- [4] Raspberry Pi Ltd, “Raspberry Pi 5,” Raspberry Pi. <https://www.raspberrypi.com/products/raspberry-pi-5/>
- [5] “Fermion: MEMS CO Gas Sensor - Detect CO Efficiently & Reliably!,” Fermion: MEMS CO Gas Sensor - Compact & Efficient Detection — DFRobot Wiki, 2025. <https://wiki.dfrobot.com/sen0564/>
- [6] “Fermion: H2S Sensor – Revolutionizing Hydrogen Sulfide Detection!,” Fermion: H2S Sensor – Efficient Hydrogen Sulfide Detection — DFRobot Wiki, 2025. <https://wiki.dfrobot.com/sen0568/> (accessed Jun. 13, 2026).
- [7] “Fermion: MEMS Methane CH4 Sensor - Precision Detection for Safety!,” Fermion: MEMS Methane CH4 Gas Detection Sensor (Breakout, 1-10000ppm) — DFRobot Wiki, 2025. <https://wiki.dfrobot.com/sen0565/> (accessed Jun. 13, 2026).
- [8] “2SMPP-03 MEMS Gauge Pressure Sensor MEMS Gauge Pressure Sensor Featuring Small Size and Low Power Consumption.” Available: https://omronfs.omron.com/en_US/ecb/products/pdf/en-2smpp-03.pdf
- [9] “2SMPP-03 MEMS Gauge Pressure Sensor MEMS Gauge Pressure Sensor Featuring Small Size and Low Power Consumption.” Available: https://omronfs.omron.com/en_US/ecb/products/pdf/en-2smpp-03.pdf

- [10] TE Connectivity, “MS5611 barometric pressure sensor datasheet,” datasheet, year.
- [11] Occupational Safety and Health Administration (OSHA), “Hydrogen Sulfide - Hazards — Occupational Safety and Health Administration,” www.osha.gov, 2024.
- [12] Firefighter Insider, “Is Methane Flammable?” <https://firefighterinsider.com/methane-flammable/>
- [13] Mining Safety and Health Association (MSHA), “Heat Stress in Mining Safety Manual Series SM 6,” 2012. Available: <https://www.msha.gov/sites/default/files/Alerts%20and%20Hazards/Heat%20Stress.pdf>
- [14] Coward, Cameron, “Aaed Musa’s Capstan Drive-Equipped Robot Dog Amazes Us,” *Hackster.io*, 2025, <https://www.hackster.io/news/aaed-musa-s-capstan-drive-equipped-robot-dog-amazes-us-9ae9cea22714>. Accessed 7 May 2026.
- [15] M. Harris, “Which Capacitor Types Should You Use?,” Altium, Oct. 07, 2020. <https://resources.altium.com/p/which-type-capacitor-should-you-use>
- [16] Ti.com, 2026. <https://www.ti.com/general/docs/suppproductinfo.tsp?distId=10&gotoUrl=https%3A%2F%2Fwww.ti.com%2Flit%2Fgpn%2F1m134>(accessed Jun. 13, 2026).
- [17] “INAx126 MicroPower Instrumentation Amplifiers.” Accessed: Jun. 13, 2026. [Online]. Available: https://www.ti.com/lit/ds/symink/ina126.pdf?HQS=dis-dk-null-digikeymode-dsf-pf-null-ww&ts=1781327774559&ref_url=https%253A%252F%252Fwww.ti.com%252Fgeneral%252Fdocs%252Fsuppproductinfo.tsp%253FdistId%253D10%2526gotoUrl%253Dhttps%253A%252F%252Fwww.ti.com%252Flit%252Fgpn%252Fina126